IPre Sueño-Vigilia y Compositional Learning

(IMT2985)

Kevin Cardemil Ulloa
kevin.cardemil@uc.cl

Contenidos

I Contenidos Preliminares 4

1.1. Reinforcement Learning 4
Una Introducción al Reinforcement Learning 4
Elementos del Reinforcement Learning 4
Procesos de Decisión de Markov Finitos 5

1.2. Complejidad de Kolmogorov 8

1.3. Minimum Description Length (MDL) 14

1.4. Backpropagation 16

1.5. Conceptos Bayesianos 17
Aplicación de la Prior y la Posterior 18

1.6. An Introduction to Variational Autoencoders 19
Introducción 19
Variational Autoencoders 21
Glosario 26

II Representaciones Latentes 27

2.1. Neural Discrete Representation 27
VQ-VAE 27
Glosario 29

2.2. Categorical Reparameterization with Gumbel-Softmax 30
La Distribución Gumbel-Softmax 30
Resultados Experimentales 32

2.3. Posterior Collapse 33
Posterior Collapse as a Phase Transition in Variational Autoencoders 33
Posterior Collapse and Latent Variable Non-identifiability 35
VQ-VAE y Posterior Collapse 36

III Síntesis de Programas 37

3.1. DreamCoder 37
Síntesis Inductiva de Programas 37
Wake-Sleep Algorithm 38

3.2. Searching Latent Program Spaces 40
Background 40
Arquitectura LPN 41
Experimentos y Resultados 44
Limitaciones 46
Preguntas 46
Glosario 46

3.3. Gradient-Based Program Synthesis with Neurally Interpreted Languages 47
Formalización 47
Inferencia Discreta Secuencial 47
Preguntas 52
Glosario 52

3.4. El Aporte de la Discretización 52
Hipótesis 53

IV Algoritmos para Redes Neuronales 54

4.1. Flat Minima 54
Tarea y Arquitectura 54
Algoritmo FMS 54
Notas Adicionales 56

V Misceláneos 58

5.1. DreamerV3 58
Algoritmo de Aprendizaje 58
Resultados 60

VI Apéndice 61

6.1. Relaciones entre Papers 61

6.2. Métodos para Entrenar una Red Neuronal 62

I Contenidos Preliminares

1.1. Reinforcement Learning

Esta sección está basada en los capítulos 1 y 3 del libro *Reinforcement Learning: An Introduction* de Richard S. Sutton y Andrew G. Barto (2014-2015).

Una Introducción al Reinforcement Learning

El **Reinforcement Learning** (RL) es un acercamiento *computacional* al *aprendizaje por interacción*, el cual pareciera subyacer a cualquier teoría del aprendizaje e inteligencia. La guía de este tipo de aprendizaje son las metas o retornos que, por lo general, son a largo plazo.

Por un lado, es diferente al **Aprendizaje Supervisado** (*Supervised Learning*), el cual consiste en aprender, a partir de un conjunto de entrenamiento etiquetado, la manera de clasificar o regresionar los nuevos ejemplos, sin embargo, no se basa en la interacción con el ambiente, como sí lo hace el RL.

Por otro lado, el RL es diferente al **Aprendizaje No Supervisado** (*Unsupervised Learning*), el cual consiste —en esencia— en encontrar patrones o *estructura* en un conjunto de entrenamiento sin etiquetas, no obstante, el RL no busca una estructura, sino que maximizar el retorno.

De esta manera, este modelo de aprendizaje lo asumiremos como un tercer paradigma del **Aprendizaje de Máquina** (*Machine Learning*).

Un reto que aparece en el RL es el tradeoff entre *explorar* y *explotar*: explorar para realizar mejores acciones a futuro; explotar decisiones que se sabe que son provechosas.

Finalmente, el RL es parte de una tendencia en la IA de retorno a principios generales y simples. Los métodos basados en principios generales se conocen como **métodos débiles** («*weak methods*»), mientras que aquellos basados en conocimiento específico se denominan **métodos fuertes** («*strong methods*»).

Elementos del Reinforcement Learning

El ciclo de aprendizaje en el RL tiene dos actores principales: el **agente** y el **ambiente**. En esencia, el agente toma decisiones buscando una manera de actuar dado cierto estado y el ambiente, el objeto con el que el agente interactúa, se encarga de indicarle una recompensa dado su actuar y el siguiente estado en que estará.

De manera más técnica, un sistema de RL se compone de: una *política*, una *señal de recompensa*, una *función de valor* y, opcionalmente, un *modelo del ambiente*.

Una **política** (*policy*) es un mapeo (que denota una probabilidad) que el agente aprenderá y que define la manera en que el learner se comporta en determinado instante. Corresponde a una especie de reglas estímulo-respuesta. En general, estos mapeos son estocásticos.

Una **señal de recompensa** (*reward signal*) es el *reward* que entrega el ambiente al agente en cada paso de tiempo. Ella afecta directamente la manera en que el agente se comportará. Por lo general, el objetivo del agente es *maximizar la recompensa total*, a largo plazo.

La **función de valor** (*value function*) especifica qué es lo bueno a largo plazo, indicando el monto total que puede esperar el agente a largo plazo. Es justamente parte de lo que queremos aprender.

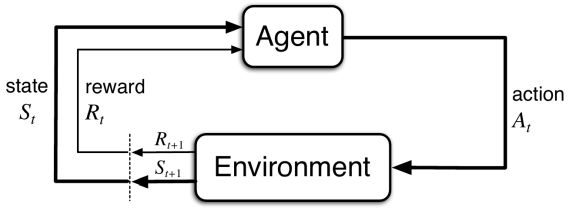
El **modelo del ambiente** es un objeto que imita el comportamiento del ambiente y que permite realizar inferencias de cómo el ambiente se comportará. En el RL podemos encontrar métodos **basados en modelos** (*model-based*), que son justamente aquellos que tienen un modelo del ambiente, y otros métodos más simples, denominados métodos **libres de modelo** (*model-free*).

Procesos de Decisión de Markov Finitos

Este problema define el campo del RL.

Una especificación completa de un ambiente define una **tarea**, esto es, una instancia de RL.

A continuación se empieza a introducir notación. El agente y el ambiente interactúan en instantes de tiempo discretos, $t = 0, 1, 2, \ldots$ (aunque existen ideas que consideran el tiempo continuo). En cada instante, el agente recibe una **representación del estado del ambiente**, $S_t \in \mathcal{S}$, donde $\mathcal{S}$ es el **espacio de estados** (*state space*). En esa situación, escoge una **acción** $A_t \in \mathcal{A}(S_t)$, donde $\mathcal{A}(S_t)$ corresponde al conjunto de acciones posibles dado el estado S_t. Como consecuencia, un instante de tiempo después, el ambiente retorna una **recompensa** $R_{t+1} \in \mathcal{R}$, donde $\mathcal{R} \subset \mathbb{R}$ es un conjunto de posibles recompensas. Además, el ambiente le indica el nuevo estado S_{t+1} al agente.



La política del agente la denotamos como $\pi_t(a|s)$, entendida como la probabilidad de que $A_t = a$ dado el estado $S_t = s$. Así, los métodos de RL consistirán en especificar cómo el agente modifica su política a partir de los resultados que obtiene.

Nota 1.1.1

> La frontera entre agente y ambiente suele estar más cerca al agente de lo que se cree. Por ejemplo, los motores y sensores de un robot se consideran parte del ambiente (pues pueden entenderse como estados y recompensas), no del agente. Normalmente, basta con preguntarse si el agente *no* puede cambiar algo arbitrariamente. De ser así, entonces ese algo no pertenece al agente. De esta manera, la frontera agente-ambiente corresponde al límite de control del agente, no de su conocimiento.

La recomenpensa es la manera en que comunicamos *qué* es lo que queremos, no *cómo* lo queremos.

En general, buscamos maximizar el **retorno esperado**, esto es, la esperanza de G_t, una función de los retornos. Por ejemplo,

$$G_t = \sum_{j=t+1}^{T} R_j,$$

donde T es el último paso, si es que tiene sentido que haya un útlimo paso. En dicho caso, cada **episodio** (o *intentos*) termina en un estado especial llamado **estado terminal**. Denotaremos con $\mathcal{S}$ los estados no-terminales y con $\mathcal{S}^+$ todos los estados, incluyendo a los terminales.

Cuando la interacción agente-ambiente no se puede separar en episodios identificables, es decir, no tenemos la noción de «último paso», llamamos a estas tareas *continuing tasks*. En este caso, tendríamos $T = \infty$ para G_t, por lo que podría diverger.

Para que G_t tenga un valor finito, se introduce el factor $0 \leq \gamma \leq 1$ conocido como **tasa de descuento** para definir el **retorno descontado**

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Unificación de Notación entre Tareas Episódicas y Continuas

Para tareas episódicas, el estado que se tiene en el instante t y episodio i se denota $S_{t,i}$. Similarmente ocurre con $A_{t,i}$, $R_{t,i}$, $\pi_{t,i}$, T_i. Como muchas veces estamos trabajando dentro de un episodio, se suele omitir el subdíndice i.

Además, para unificar la notación del retorno, se pensará que las tareas episódicas tienen infinitos pasos, pero que al llegar al estado terminal se quedan en un *estado absorbente* que transiciona sólo a si mismo con retorno cero.

Procesos de Decisión de Markov

Si el espacio de estados y el de acciones son finitos, entonces estamos ante un **finite MDP**. Un finite MDP se definte completamente a partir de: el espacio de estados, el espacio de acciones y por las dinámicas paso a paso del ambiente.

Dado un estado s y una acción a, la probabilidad de llegar al estado s' y obtener una recompensa r se denota por

$$p(s', r|s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}.$$

Al tener todos los valores de p dadas las posibles combinaciones de s', t, s, a, se tienen definidas las *dinámicas del ambiente*.

Dadas estas dinámicas, se puede calcular lo que sea. Por ejemplo, los *retornos esperados de un par estado-acción*,

$$\begin{aligned} r(s, a) &= \mathbb{E}(R_{t+1} \mid S_t = s, A_t = a) \\ &= \sum_{r \in \mathcal{R}} r \cdot \Pr(R_{t+1} = r | S_t = s, A_t = a) \\ &= \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r|s, a). \end{aligned}$$

Así también se pueden tener las *probabilidades estado-transición* (es decir, la probabilidad de pasar de un estado a otro tomando cierta acción),

$$p(s'|s, a) = \Pr(S_{t+1} = s' \mid S_t = S, A_t = a) = \sum_{r \in \mathcal{R}} p(s', r|s, a).$$

Finalmente, también se pueden obtener los *retornos esperados para un triple estado-acción--siguiente estado*,

$$
\begin{aligned}
r(s,a,s') &= \mathbb{E}(R_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s') \\
&= \sum_{r\in\mathcal{R}} r \cdot \Pr(R_{t+1} = r \mid S_t = s, A_t = a, S_{t+1} = s') \\
&= \sum_{r\in\mathcal{R}} r \cdot \frac{\Pr(R_{t+1} = r, S_t = s, A_t = a, S_{t+1} = s')}{\Pr(S_t = s, A_t = a, S_{t+1} = s')} \\
&= \sum_{r\in\mathcal{R}} r \cdot \frac{\Pr(R_{t+1} = r, S_{t+1} = s' \mid A_t = a, S_t = s)\cancel{\Pr(A_t = a, S_t = s)}}{\Pr(S_{t+1} = s' \mid A_t = a, S_t = s)\cancel{\Pr(A_t = a, S_t = s)}} \\
&= \sum_{r\in\mathcal{R}} r \cdot \frac{p(s', r|s, a)}{p(s'|a, s)}.
\end{aligned}
$$

Para los MDP finitos podemos construir un **grafo de transición** que permiten resumir las dinámicas visualmente. En ellos existen dos tipos de nodos: los **nodos de estado**, de los cuales se construye uno por cada estado $S \in \mathcal{S}$; y los **nodos de acción**, de los cuales se construye uno por cada par estado-acción. Los nodos de acción tienen entrante una arista dirigida desde un nodo de estado y tienen saliente una (o más) arista(s) dirigida(s) a otro nodo(s) de estado, donde este tipo de aristas presentas anotada la probabilidad y recompensa correspondiente de la transición estado-acción-estado siguiente.

Funciones de Valor

Por hacer en el futuro

1.2. Complejidad de Kolmogorov

Buscamos describir un objeto en una string binaria. Un objeto podría tener muchas descripciones, pero una descripción sólo puede corresponder a un único objeto. Además, podríamos tomar el largo de la descripción más corta y considerarla como la *complejidad* del objeto.

Pero en ese caso podríamos caer en una trampa descrita en la **Richard-Berry paradox**, donde definimos un número natural como «el menor natural que no puede ser descrito en menos de veinte palabras». Si ese número existe, lo acabamos de describir en 13 palabras, contradiciendo la frase que lo define. Si tal número no existe, todos los números pueden describirse en menos de veinte palabras. Este resultado aparece por la *ambigüedad* sistemática de la palabra «descrito» (o, en otros casos, de «definible»). Resolver este tipo de paradojas requiere indicar dónde exactamente nuestro lenguaje «anduvo mal» e indicar restricciones para evitarlas.

Asuma que cada descripción describe a lo sumo un objeto. Así, existe un **método de especificación** $D : Y \to X$ que asocia, con una descripción $y \in Y$, a lo sumo un objeto $x \in X$. Además, para que las descripciones sean útiles, las asumiremos finitas, así que existe una cantidad numerable de descripciones y, por tanto, a lo sumo una cantidad numerable de objetos descriptibles.

Para medir la complejidad de un objeto $x \in X$, podríamos escoger aquel que tenga menor costo de transmisión, esto es, el menor largo de y tal que $D(y) = x$. Sin embargo, la complejidad de un objeto dependería directamente de D. Es por esto que, para poder comparar la complejidad de objetos, haremos que la complejidad de x dependa únicamente de él.

Además, para que las especificaciones D sean útiles, será necesario que $D(y)$ sea ejecutable en una **manera efectiva**, es decir, que al menos puede ser ejecutable por humanos o por máquinas.

Asumiremos como resultado que el conjunto de funciones parciales computables contiene una función óptima que minimiza el largo de descripción de cualquier otra función. Denotaremos dicha función con D_0. Y para cualquier otra función computable D, para cualquier objeto x, existe una descripción de x bajo D_0 que es más corta —hasta una constante aditiva independiente de x— que cualquier otra descripción de x bajo D.

Identificaremos el largo de descripción de x con respecto a una función de especificación fijada D_0 como la **complejidad algorítmica** de x. La optimalidad de D_0 en este sentido quiere decir que la complejidad de un objeto es **invariante** (hasta una constante aditiva independiente de x) bajo la transición de un método de especificación a otro. Este resultado se conoce como

Teorema 1.2.1 de Invarianza

Existe una máquina de Turing universal (o función parcial computable universal) U tal que, para cualquier otra máquina de Turing (o función computable) F, existe una constante c_F que depende exclusivamente de F y de la elección de U, de modo que para cualquier cadena x,

$$C_U(x) \leq C_F(x) + c_F,$$

donde

- $C_M(x) = \min\{l(y) : D(y) = x\}$ denota la complejidad de Kolmogorov en la máquina o función $D : Y \to X$,
- y es la descripción,
- $l(y)$ es la longitud de la descripción.

De esta manera, la complejidad de un objeto es un atributo objetivo e inherente a él.

Notación

Si ϕ es una **función parcial** de A a B, entonces para cada $x \in A$, o bien $\phi(x) \in B$, o bien $\phi(x)$ está indefinida. Si $\phi(x) < \infty$, diremos que ϕ **converge**. Por el contrario, si $\phi(x) = \infty$, diremos que ϕ **diverge** o que está **indefinida** en x. Si ϕ converge para cada $x \in A$, entonces diremos que es una **función total**, de otro modo diremos que es una **función estrictamente parcial**. Si cada miembro de B está también en el rango de ϕ, entonces diremos que ϕ **mapea sobre (*onto*)** B, si no **mapea a (*into*)** B. Además, si para cada $x \neq y$, ϕ converge y $\phi(x) \neq \phi(y)$, entonces llamamos a φ un **mapeo uno-a-uno**, de otro modo la llamamos un **mapeo muchos-a-uno**.

Tendremos los siguientes cuantificadores:

- $\exists$: Existe al menos un elemento.
- $\forall$: Para todo elemento.
- $\exists^\infty$: Existen infinitos elementos.
- $\forall^\infty$: Para todo elemento, excepto para finitos.

$$\neg\exists^\infty x.\phi(x) \iff \forall^\infty x.\neg\phi(x).$$

Nota que podemos hacer el símil entre $\exists^\infty$ con *infinitely often* y entre $\forall^\infty$ con *almost always*.

Strings Binarias

Tendremos nuestro conjunto no vacío $\mathcal{B}$ de **elementos básicos**. Usualmente, $\mathcal{B} = \{0, 1\}$. El conjunto de todas las strings finitas sobre $\mathcal{B}$ es

$$\mathcal{B}^* := \{\epsilon, 0, 1, 00, 01, 10, 11, ...\},$$

donde ϵ denota la **string vacía**.

La **concatenación** es una operación binaria de elementos en $\mathcal{B}^*$ que toma dos elementos $x, y \in \mathcal{B}^*$ y los lleva a $xy \in \mathcal{B}^*$. Algunas propiedades son:

1. $\mathcal{B}^*$ es cerrada bajo concatenación.
2. La concatenación es asociativa.
3. La concatenación tiene un elemento *unitario* ϵ:

$$\epsilon x = x\epsilon = x.$$

Definiremos una **correspondencia** esencial entre los naturales $\mathbb{N}$ y las strings binarias finitas $\mathcal{B}^*$. Es de notar que no usaremos la representación binaria estándar, pues tiene la desventaja de que, o bien hay strings que no representan números naturales, o bien hay números naturales con múltiples representaciones. Por ejemplo, es bastante conocido que los ceros como prefijos no tienen significado en este sistema: ¡010, 10 y 0010 representan

al mismo número! Por ello, definimos un mapeo uno-a-uno de $\mathcal{B}^*$ sobre $\mathbb{N}$ asociando cada string con su índice en el **orden lexicográfico por longitud creciente (*shortlex*)**

$$\begin{aligned}\epsilon &\longleftrightarrow 0\\ 0 &\longleftrightarrow 1\\ 1 &\longleftrightarrow 2\\ 00 &\longleftrightarrow 3\\ 01 &\longleftrightarrow 4\\ 10 &\longleftrightarrow 5\\ 11 &\longleftrightarrow 6\\ &\vdots\end{aligned}$$

De esta manera, representamos a

$$x = 2^{n+1} - 1 + \sum_{i=0}^{n} a_i 2^i$$

como $a_n \cdots a_1 a_0$. Además, consideraremos al string en $\mathcal{B}^*$ y al número con que se corresponde como exactamente el mismo objeto.

Para una string x, x_i denota el i-ésimo bit y $x_{i:j}$ denota el segmento $x_i x_{i+1} \cdots x_j$ de largo $j - i + 1$. El **reverso** lo denotaremos por x^R. El **largo** $l(x)$ de la string finita lo definimos como el número de bits que contiene. En particular, $l(\epsilon) = 0$, $l(x^R) = l(x)$ y $l(xy) = l(x) + l(y)$.

Sea D una función $D : \{0,1\}^* \to \mathbb{N}$. Diremos que el dominio de D serán las **palabras código** (*code words*) y su rango el conjunto de **palabras fuente** (*source words*). De esta manera, $D(y) = x$ se interpreta como que y es una code word de la source word x y D es una **función de decodificación**. Al revés, podemos definir una **sustitución de codificación** $E = D^{-1}$ (que no necesariamente una función).

Diremos que una string x es prefijo de z si es que existe una string y tal que $z = xy$. Diremos que $A \subset \{0,1\}^*$ es **libre de prefijos** (*prefix-free*) si ningún elemento de A es prefijo de otro elemento en A.

Ejemplo 1.2.2

Si $A = \{101, 010, 11, 00\}$ es un conjunto libre de prefijos.

Una función $D : \{0,1\}^* \to \mathbb{N}$ define un **código prefijo** (*prefix-code*) si su dominio es libre de prefijos.

Ejemplo 1.2.3

Un código prefijo muy utilizado es aquel obtenido al reservar el cero como punto de detenimiento y codificando $x \in \mathbb{N}$ como $1^x 0$. Así, $D : A \to \mathbb{N}$ tiene

$$A = \{0, 10, 110, 1110, 11110, \ldots\}$$

que claramente es libre de prefijos. Además, podemos prefijar un objeto con su largo e iterar la idea para obtener códigos incluso más cortos. Para esto se define

$$E_i(x) = \begin{cases} 1^x 0 & \text{para } i = 0 \\ E_{i-1}(l(x))x & \text{para } i \geq 1. \end{cases}$$

Así, $E_0(x) = 1^x 0$ (que tiene largo $x+1$), y $E_1(x) = 1^{l(x)}0x$ (que tiene largo $2l(x)+1$). [Recuerde que $l(x) \approx \ln x$, así que asintóticamente los E_i tienen largo más corto que los anteriores.] En particular, E_1 tiene una notación especial:

$$\overline{x} := E_1(x) = 1^{l(x)}0x$$
$$l(\overline{x}) = 2l(x) + 1.$$

Llamamos a $\overline{x}$ la versión **auto-limitante** (*self-limiting*) de la string x.

Computabilidad y Máquinas de Turing

Estudiaremos que todo lo que puede ser razonablemente considerado computable por un *computador humano* (un calculista) usando un procedimiento fijo puede ser computado por una máquina hipotética, una máquina de Turing. Y, como Turing indicó, cualquier proceso que puede ser naturalmente llamado *procedimiento efectivo* es realizado por una máquina de Turing. Esto último se conoce como la **tesis de Turing**. De hecho, todos los intentos de dar una precisa aunque intuitiva definición de la noción de lo que es un «procedimiento efectivo» (en el sentido más amplio) han resultado ser equivalentes. En este sentido, la **tesis de Church** establece que existe una noción de lo que es la *computabilidad efectiva* independiente de la formalización particular.

Una **máquina de Turing** consiste en un programa finito, llamado **control finito**, capaz de manipular una **cinta**, esto es, una lista lineal de **celdas**, usando un puntero de un acceso, llamado **cabeza**. La cinta tiene dos lados, derecha e izquierda. Además, el control finito tiene un **conjunto de estados** Q finito y cada celda puede contener un 0, un 1 y un **blanco** B. El tiempo se asume discreto, partiendo con la máquina desde el instante cero. En cualquier instante la cabeza está posicionada sobre una celda y la **escanea**. En el instante cero la cabeza está sobre una **celda inicial** y el control finito está en un estado distinguido $q_0 \in Q$ y, además, todas las celdas contienen 0´s, excepto por una secuencia finita y contigua de celdas, que se extienden desde la celda inicial, las cuales contienen una secuencia binaria ingresada, llamada **input**.

El aparato puede realizar las siguientes operaciones (sólo una por instante):

1. Escribir un elemento de $A = \{0, 1, B\}$ en la celda que escanea.
2. Mover la cabeza una celda a la derecha o a la izquierda.

Al finalizar cada paso, el control finito toma un estado de Q. Además, el aparato está construido de modo que se comporte de acuerdo a una **lista de reglas** finita, las cuales indican, según el símbolo leído y el estado actual, la operación a realizar *en el siguiente instante* y el estado en que ponerse al terminar la siguiente operación. Cada regla tiene la forma

$$(p, s, a, q),$$

donde:

- p es el estado actual.
- s es el símbolo siendo escaneado.
- a es la acción a ejecutar en el siguiente paso, pudiendo escoger de $S = \{0, 1, B, L, R\}$ (esto es, la máquina o se mueve o escribe, pero no las dos al mismo tiempo).
- q es el estado a tomar al finalizar este instante.

La máquina de Turing es determinista: no pueden haber dos reglas que compartan los mismos dos elementos (es decir, p, s). Además, puede ocurrir que un estado p y un símbolo s no tengan una regla definida. En ese caso, el aparato **para** (*halts*). De esta manera, podemos definir la máquina de Turing mediante un mapeo (o bien, una función parcial) de $Q \times A$ hacia $S \times Q$.

Podemos asociar una función parcial con cada máquina de Turing:

- El input $x = x_1 ... x_n$ a la máquina se representa mediante una n-tupla $(x_1, ..., x_n)$, donde cada $x_i \in \{0, 1\}$. (En caso de que $x \in \mathbb{N}$ sea un número, primero se transforma mediante *shortlex*.)
- Se escribe el input en la cinta hacia la derecha partiendo desde la celda inicial.
- El instante en que la máquina para, el output corresponde a la secuencia de símbolos que queda escrita en la cinta. (En caso de que la máquina nunca se detenga, no se produce ninguna salida.)

Así, se ha asociado una máquina de Turing con una función parcial f en el sentido de que $f(x) = y$ siempre que se ingresa x a la máquina y esta tiene como output y y, cuando la máquina no se detiene, entonces $f(x)$ es indefinido. Por tanto, podemos decir que *toda máquina de Turing computa (o induce) una función parcial.*

Llamaremos a aquella f **función parcial computable**. Si la máquina se detiene para todos los inputs posibles, entonces diremos que f es **total y computable**, o simplemente **computable**.

Definición 1.2.4 Tesis de Church

> La clase de funciones numéricas algorítmicamente computables (en el sentido intuitivo) coincide con la clase de funciones parcialmente computables.

La tesis de Church se piensa intuitivamente como que «dada una descripción de un procedimiento en términos de un conjunto informal de instrucciones, nosotros podemos derivar un procedimiento formal en términos de máquinas de Turing».

Definición 1.2.5 Máquina de Turing Universal

> Una máquina de Turing U es **universal** si puede imitar el comportamiento de cualquier otra máquina de Turing T.

Está demostrado que dichas máquinas existen (de hecho, son infinitas) y que pueden ser efectivamente construidas. Bastaría con una descripción del programa finito de T y el input a ingresar a ella, para ingresarlos en la cinta inicial de U. Para ejecutar las aacciones que haría T, U iría leyendo el programa de T y las ejecutaría en su propia cinta.

Existe un emparejamiento (efectivo computable) uno-a-uno entre los naturales y las máquinas de Turing llamado **enumeración efectiva**, $T_1, T_2, ...$ De hecho, esta enumeración (efectiva) permite crear una enumeración efectiva de funciones parciales computables (¿Por qué?) $\phi_1, \phi_2, ...$

Existe una diferencia entre el *nombre* para ψ y una *función* ψ. Un nombre para ψ puede ser un algoritmo que la calcule con una máquina de Turing, o puede ser un **índice** $i \in \mathbb{N}$ tal que ψ es igual a ϕ_i es la enumeración efectiva. [Cada función parcial computable ψ ocurre

muchas veces dada una enumeración efectiva, pues podemos construir distintas máquinas de Turing que para el mismo input produzcan el mismo output.]

Definición 1.2.6

> La función parcial computable $\nu^{(2)}(i, x)$ computada por la máquina universal U se llama la **función parcial universal computable**.

Un conjunto $A \subseteq \mathbb{N}$ es **computablemente enumerable** (c.e.) si es vacío o es el rango de alguna función f total computable. En ese caso, diremos que f **enumera** a A. Diremos además que un conjunto $B \subseteq \mathbb{N}$ es **co-computablemente enumerable** (co-c.e.) si su complemento $A = \overline{B}$ es c.e.. Un conjunto $C \subseteq \mathbb{N}$ es **computable** ssi su función característica es total computable, esto es, $f = \mathbb{1}_A$ es total computable. Así que todos los conjuntos computables son c.e. y co-c.e..

1.3. Minimum Description Length (MDL)

El material de este capítulo está inspirado en las notas Aprendizaje Composicional y Generalización OOD de MAT2320 que me compartió el profesor Petrache.

A continuación, profundizamos de una manera más rigurosa la 1.2. Complejidad de Kolmogorov y presentamos el Minimum Description Length (MDL).

Sea $\mathcal{U}$ una máquina de Turing universal fija.

Definición 1.3.1 Complejidad de Kolmogorov

Sean $x, y \in \{0,1\}^*$ cadenas binarias. Luego, la **complejidad de Kolmogorov de x dado y** se define como

$$K(x|y) := \min\{|p| : \mathcal{U}(p, y) = x\},$$

donde p es un programa (finito, como siempre) que produce x a partir de y en $\mathcal{U}$ y $|p|$ denota su longitud en bits.

La **complejidad incondicional de x** es

$$K(x) := K(x|\varepsilon),$$

donde ε es la cadena vacía en $\{0,1\}^*$.

Intuitivamente, $K(x|y)$ mide cuánta información de x falta dado que se conoce y, mientras que $K(x)$ mide el tamaño de la descripción más corta de x (si no conozco nada de x...).

El teorema de invarianza indica que la complejidad de un x *esencialmente* no depende la elección de $\mathcal{U}$.

Teorema 1.3.2 Teorema de Invarianza

Para cualesquiera dos máquinas de Turing universales $\mathcal{U}$ y $\mathcal{V}$, existe una constante $c_{\mathcal{U},\mathcal{V}} \geq 0$ tal que

$$|K_{\mathcal{U}}(x) - K_{\mathcal{V}}(x)| \leq c_{\mathcal{U},\mathcal{V}}, \quad \forall x \in \{0,1\}^*.$$

Otro teorema interesante.

Teorema 1.3.3

La función $x \mapsto K(x)$ es no computable.

Este teorema, en nuestro enfoque, implica que, como $K(x)$ no es computable, en general no podamos asegurar haber encontrado la representación óptima de x, puesto que cualquier método computable entregará un largo para el que, en algunos casos, $K(x)$ será menor (si no, K sería computable).

Ahora presentamos una aplicación práctica de la complejidad de Kolmogorov.

Definición 1.3.4 Minimum Description Length (MDL)

Sea $\mathcal{H}$ una clase de hipótesis y D un conjunto de datos. El **principio de Longitud Mínima de Descripción** indica que se seleccione la hipótesis

$$h^* = \arg\min_{h \in \mathcal{H}}(L(h) + L(D|h)),$$

donde $L(h)$ es la longitud en bits de la descripción de h y $L(D|h)$ es la longitud de la descripción de D dado h.

El MDL, esencialmente, formaliza la navaja de Occam, indicando que el criterio para escoger una hipótesis consiste en elegir aquella que minimiza la longitud de la hipótesis sumada a la de los datos codificados/descritos desde nuestra hipótesis:

- Una hipótesis muy simple, tiene $L(h)$ pequeño, pero $L(D|h)$ grande.
- Una hipótesis muy compleja, tiene $L(h)$ grande, pero $L(D|h)$ pequeño (en caso de que sea sobreajustada).

Nota 1.3.5

El MDL descrito anteriormente se conoce como MDL «crudo» o «de dos partes». Existe otro MDL, llamado «refinado», que utiliza códigos universales en vez de dos longitudes por separado.

1.4. Backpropagation

Por hacer en el futuro

Backpropagation es un algoritmo para calcular gradientes de manera eficiente y se suele ocupar para entrenar redes neuronales, parametrizadas por $f(x; w)$, donde w son los pesos, pues muchas veces se encuentran los pesos usando descenso de gradiente sobre el error $E(w)$ (que suele depender de una función de pérdia $\ell(x; w)$), iterando

$$w \leftarrow w - \eta \nabla_w E(w),$$

donde $\eta > 0$ es el **learning rate**.

El Backpropagation es una aplicación de programación dinámica.

Nota 1.4.1

> En el Apéndice, en 6.2. Métodos para Entrenar una Red Neuronal, se encuentra un mapa conceptual que muestra más métodos con los que se puede entrenar una red neuronal.

1.5. Conceptos Bayesianos

A lo largo de diversas investigaciones se usan conceptos bayesianos, como prior y posterior, por lo que consideré importante repasarlos como una base para el aprendizaje composicional.

Sea $\boldsymbol{z}$ una cantidad desconocida (como podría una variable latente o un programa) y sea $\boldsymbol{x}$ un dato. El teorema de Bayes establece una igualdad para la distribución condicional de $\boldsymbol{z}|\boldsymbol{x}$:

$$p(\boldsymbol{z}|\boldsymbol{x}) = \frac{p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p(\boldsymbol{x})}, \tag{1}$$

donde:

- $p(\boldsymbol{z}|\boldsymbol{x})$ es lo que se conoce como **posterior**. Corresponde a, dado los datos, qué tan probable creemos que es cierto valor de $\boldsymbol{z}$ (o, más rigurosamente, cierto rango de valores).
- $p(\boldsymbol{x}|\boldsymbol{z})$ es la **verosimilitud**. Corresponde a qué tan probable es el dato que se tiene asumiendo cierto valor de $\boldsymbol{z}$.
- $p(\boldsymbol{z})$ es la **prior**. Corresponde a la distribución, previo a la experiencia, esto es, previo a tener $\boldsymbol{x}$, que creemos que tienen los valores de $\boldsymbol{z}$. Cuando no tenemos una creencia previa para $\boldsymbol{z}$, se deja como una uniforme.
- $p(\boldsymbol{x})$ es la **evidencia**. Corresponde a la distribución que creemos que tienen los datos.

Nota 1.5.1 La posterior es una mezcla

> Observa cómo es que en la ecuación (1) $p(\boldsymbol{z}|\boldsymbol{x}) \propto p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})$, así que la posterior es una mezcla de lo que creíamos que valdría $\boldsymbol{z}$ (con la prior) y la información que nos entregó $\boldsymbol{x}$.

Nota 1.5.2 Relación con el *posterior collapse*

> En caso de que $\boldsymbol{x}$ no nos informe nada en nuestro modelo, la posterior y la prior se vuelven idénticas, que es justo el caso del *posterior collapse*. De hecho, en ese caso, $p(\boldsymbol{x}|\boldsymbol{z}) = p(\boldsymbol{x})$ y $\boldsymbol{x}$ y $\boldsymbol{z}$ se vuelven independientes.

Muchas veces convendrá además tener en mente el modelo generativo, que aprovecha la siguiente igualdad:

$$p(\boldsymbol{x}, \boldsymbol{z}) = p(\boldsymbol{z})p_\theta(\boldsymbol{x}|\boldsymbol{z}),$$

donde θ es un parámetro del modelo generativo. Se entiende como que:

1. Se muestrea un $\boldsymbol{z} \sim p(\boldsymbol{z})$.
2. Se genera un $\boldsymbol{x} \sim p_\theta(\boldsymbol{x}|\boldsymbol{z})$.

A partir de esta ecuación, se obtiene que

$$p(\boldsymbol{x}) \overset{\text{distribución marginal}}{=} \int p(\boldsymbol{x}, \boldsymbol{z})\,\mathrm{d}\boldsymbol{z} = \int p(\boldsymbol{z})p_\theta(\boldsymbol{x}|\boldsymbol{z})\,\mathrm{d}\boldsymbol{z}.$$

Por lo tanto, la distribución de los datos no depende de $\boldsymbol{z}$.

Aplicación de la Prior y la Posterior

El método de **Estimadores Máximo-Verosímiles** (EMV, o MLE en inglés) se escoge aquel $\hat{\boldsymbol{z}}_{\text{EMV}}$ tal que

$$\hat{\boldsymbol{z}}_{\text{EMV}} = \arg\max_{\boldsymbol{z}} p(\boldsymbol{x}|\boldsymbol{z}) = \arg\max_{\boldsymbol{z}} [\log p(\boldsymbol{x}|\boldsymbol{z})],$$

es decir, aquel que maximiza la verosimilitud de los datos observados.

Un método similar es **máximo a posteriori** (MAP, *Maximum A Posteriori*), que consiste en escoger aquel $\hat{\boldsymbol{z}}_{\text{MAP}}$ tal que

$$\hat{\boldsymbol{z}}_{\text{MAP}} = \arg\max_{\boldsymbol{z}} [\log p(\boldsymbol{x}|\boldsymbol{z}) + \log p(\boldsymbol{z})],$$

esto es, es similar al EMV, sólo que también considera la creencia que tenemos sobre los $\boldsymbol{z}$ previo a la experiencia. (Si $p(\boldsymbol{z})$ es uniforme, el método se convierte en MLE.)

1.6. An Introduction to Variational Autoencoders

Nota 1.6.1 Fuente

> Contenido estudiado de *«An Introduction to Variational Autoencoders»* de D. P. Kingma y M. Welling (2019). [arXiv:1906.02691]

Introducción

Los Variational Autoencoders (VAE) entregan un framework para aprender modelos profundos de variables latentes y los modelos de inferencia correspondientes.

Suponga que queremos modelar la distribución conjunta de unas variables observadas. Sea $\boldsymbol{x}$ un vector de estas variables. De hecho, asumiremos que $\boldsymbol{x}$ es una muestra de un *proceso desconocido y subyacente*, con distribución real $p^*(\boldsymbol{x})$ desconocida. Así, lo que buscamos es aproximar esta p^*.

Para aproximar p^*, partimos de un modelo $p_{\boldsymbol{\theta}}$, cuyos parámetros en $\boldsymbol{\theta}$ ajustaremos de modo que

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) \approx p^*(\boldsymbol{x}).$$

De esta manera, tendremos un modelo generativo tal que

$$\boldsymbol{x} \sim p_{\boldsymbol{\theta}}(\boldsymbol{x}).$$

Modelos Condicionales

A veces tendremos conocimiento sobre la distribución de los datos, lo que prodremos incluir con una distribución *a priori*. Los modelos que involucren este tipo de conocimiento se conocen como **Modelos Condicionales**. Asuma que queremos aproximar una distribución sobre los valores de una variable $\boldsymbol{y}$, dados unos datos $\boldsymbol{x}$ (observados, el **input**). Es decir, queremos aproximar $p^*(\boldsymbol{y}|\boldsymbol{x})$ ajustando $p_{\boldsymbol{\theta}}(\boldsymbol{y}|\boldsymbol{x})$.

Ejemplo 1.6.2

> Un ejemplo clásico es la clasificación de imágenes. Una imagen se toma como input, $\boldsymbol{x}$, y su clase se piensa como valor de $\boldsymbol{y}$.

Redes Neuronales

Asumiremos que las **redes neuronales** son redes feed-forward diferenciables. Dos características de ellas son que:

(1) Permiten *aproximar funciones*, en particular, densidades.

(2) Son *escalables* a grandes modelos y grandes datasets, pues admiten optimización con SGD.

Cuando estas redes neuronales tienen muchas capas ocultas, decimos que estamos ante un problema de **deep learning**. Denotaremos estas redes como la función vectorial (e.g. un vector de probabilidades de cada clase)

$$\text{NeuralNet}(\cdot).$$

Ejemplo 1.6.3

La clasificación de imágenes con redes neuronales (LeCun *et al.*, 2016) se ha realizado parametrizando una distribución categórica $p_{\boldsymbol{\theta}}(y|\boldsymbol{x})$ sobre una clase de etiquetas y, condicionado a una imagen $\boldsymbol{x}$. (Los pesos de la red neuronal corresponden a $\boldsymbol{\theta}$.) Se define

$$\boldsymbol{p} = \text{NeuralNet}(\boldsymbol{x}),$$

esto es, la salida que produce la red neuronal que, en este caso, corresponde a un vector de probabilidades de clases (ergo $\sum_i p_i = 1$ y $p_i \geq 0$). Luego, se piensa

$$p_{\boldsymbol{\theta}}(y|\boldsymbol{x}) = \text{Categorical}(y; \boldsymbol{p}),$$

donde Categorical es una distribución categórica parametrizada por $\boldsymbol{p}$.

Modelos Gráficos Dirigidos

Los **modelos probabilísticos dirigidos**, **modelos gráficos probabilísticos** o **redes bayesianas** son modelos donde todas las variables están organizadas en un digrafo acíclico:

- Los nodos representan a las variables.
- Las aristas representan relaciones de independencia.

Denotaremos con $\text{Pa}(\boldsymbol{x}_j)$ los padres de la variable $\boldsymbol{x}_j$). Aquí, la distribución sobre las variables resulta en

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}_1, ..., \boldsymbol{x}_M) = \prod_{j=1}^{M} p_{\boldsymbol{\theta}}(\boldsymbol{x}_j|\text{Pa}(\boldsymbol{x}_j)).$$

Nota 1.6.4

Si $\boldsymbol{x}_j$ no tiene padres, i.e. es una raíz, $\text{Pa}(\boldsymbol{x}_j) = \varnothing$, pero la distribución se piensa incondicional simplemente.

Dataset y Verosimilitud

Nuestro **dataset** es

$$\mathcal{D} = \left\{\boldsymbol{x}^{(i)}\right\}_{i=1}^{N} =: \boldsymbol{x}^{(1:N)}.$$

Dado un datates $\mathcal{D}$, denotamos con $N_{\mathcal{D}}$ su tamaño. Muchas veces asumimos las observaciones $\boldsymbol{x}^{(i)}$ i.i.d. Así, podemos calcular la probabilidad (densidad) de los datos según el modelo. En particular, podemos obtener la log-probabilidad asociada

$$\log p_{\boldsymbol{\theta}}(\mathcal{D}) = \sum_{i=1}^{N} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}^{(i)}).$$

El criterio más común para los modelos es el de la **Máxima Verosimilitud (ML)**, esto es, encontrar el $\boldsymbol{\theta}$ que maximiza la verosimilitud (o la log-verosimilitud). (Que es equivalente a la minimización de la divergencia de Kullback-Leibler). Para ello, se usa **Stochastic Gradient Descent (SGD)**[1] que usa minibatches $\mathcal{M} \subset \mathcal{D}$ y define el estimador

[1] Podríamos usar **Batch Gradient Descent** considerando al maximizar siempre todo $\mathcal{D}$, pero el costo sería muy alto para datasets grandes (lineal).

$$\frac{1}{N_{\mathcal{M}}} \log p_{\boldsymbol{\theta}}(\mathcal{M}) \simeq \frac{1}{N_{\mathcal{D}}} \log p_{\boldsymbol{\theta}}(\mathcal{D})$$

(«$\simeq$» indica que un miembro de la ecuación es un estimador insesgado del otro). Este estimador es diferenciable, por lo que podemos estimar

$$\frac{1}{N_{\mathcal{M}}} \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\mathcal{M}) \simeq \frac{1}{N_{\mathcal{D}}} \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\mathcal{D}).$$

Variables Latentes y Modelos Profundos Latentes (DLVMs)

El método presentado en la sección anterior es válido cuando las variables del modelo dirigido son *totalmente observadas*, pero podemos extenderlo a modelos dirigidos con variables latentes. Las **variables latentes** (denotadas con $\boldsymbol{z}$) son variables que son parte del modelo, pero que no observamos, por tanto, no forman parte del dataset.

En modelamiento incondicional, la distribución conjunta es $p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z})$. Para conocer $p_{\boldsymbol{\theta}}(\boldsymbol{x})$,

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z}) \, \mathrm{d}\boldsymbol{z} = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) p_{\boldsymbol{\theta}}(\boldsymbol{z}) \, \mathrm{d}\boldsymbol{z}.$$

(Recuerda que $p_\theta(\boldsymbol{x}, \boldsymbol{z}) = p_\theta(\boldsymbol{z}|\boldsymbol{x}) p_\theta(\boldsymbol{x}) = p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}) p_\theta(\boldsymbol{z})$.)

Cuando la distribución de un modelo de variable latente, $p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z})$, está parametrizada por una red neuronal, usamos el término **Modelo Profundo de Variable Latente** (*deep latent variable model, DLVM*). Una ventaja de los DLVMs es su expresividad: la distribución marginal de $\boldsymbol{x}$ puede ser muy compleja, aun cuando la distribución condicional $p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$ o la *a priori* $p_{\boldsymbol{\theta}}(\boldsymbol{z})$ son simples.

Intratabilidades

La principal dificultad de los DLVMs es que la marginal $p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \int p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z}) \, \mathrm{d}\boldsymbol{z}$ es intratable, pues no tiene una fórmula analítica ni un estimador eficiente. Por esto, no podemos diferenciar respecto a $\boldsymbol{\theta}$ para optimizarla (que es nuestro objetivo), esto es, no podemos aplicar SGD.

La intratabilidad de $p_{\boldsymbol{\theta}}(\boldsymbol{x})$, recordando que

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z}) = p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}) p_{\boldsymbol{\theta}}(\boldsymbol{x}),$$

Provoca que $p(\boldsymbol{z}|\boldsymbol{x})$ sea intratable, en principio. Es decir, ambas densidades son intratables en DLVMs.

Para resolver esto, las *técnicas aproximadas de inferencia* nos permitirán aproximar $p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ y $p_{\boldsymbol{\theta}}(\boldsymbol{x})$ en los DLVMs. Los métodos tradicionales son relativamente caros.

Variational Autoencoders

Superaremos las intratabilidades de los DLMVs con los VAEs.

Encoder

Definimos el **encoder**, **modelo de reconocimiento** o **modelo de inferencia** $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$. Aquí $\boldsymbol{\phi}$ indica los parámetros del modelo, llamados **parámetros variacionales**. Buscaremos que esta distribución sea lo más parecida a la posteriori $p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$, es decir,

$$q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \approx p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}).$$

Con ella podremos más adelante optimizar $p_{\boldsymbol{\theta}}(\boldsymbol{x})$.

El modelo de inferencia puede ser prácticamente cualquier modelo gráfico dirigido,

$$q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) = q_{\boldsymbol{\phi}}(\boldsymbol{z}_1, ..., \boldsymbol{z}_M|\boldsymbol{x}) = \prod_{j=1}^{M} q_{\boldsymbol{\phi}}(\boldsymbol{z}_j|\mathrm{Pa}(\boldsymbol{z}_j), \boldsymbol{x}).$$

Además, $q_{\boldsymbol{\phi}}$ puede ser parametrizada por una red neuronal profunda.

Ejemplo 1.6.5

Podríamos tener

$$q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) = \mathrm{Normal}(\boldsymbol{z}; \boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma})),$$

donde

$$(\boldsymbol{\mu}, \log \boldsymbol{\sigma}) = \mathrm{EncoderNeuralNet}_{\boldsymbol{\phi}}(\boldsymbol{x}).$$

Usaremos un único encoder neuronal para realizar inferencia posterior sobre *todos* los puntos del dataset, a diferencia de lo que ocurre con otros métodos de inferencia variacional, donde los parámetros de $\boldsymbol{\phi}$ se optimizan separadamente *para cada punto* del dataset. Nuestra técnica se conoce como **inferencia variacional amortizada**. Esto nos permitirá ser más eficientes y aprovechar el SGD.

Evidence Lower Bound (ELBO)

Para cada elección de encoder $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$ (con $\boldsymbol{\phi}$ ya fijo), **REVISAR**

$$\begin{aligned}
\mathbb{E}(\log p_{\boldsymbol{\theta}}(\boldsymbol{x})) &= \int q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \cdot \log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{z} \\
&= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x})] \\
&= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log\left(\frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right)\right] \\
&= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log\left(\frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z})}{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})} \cdot \frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right)\right] \\
&= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log\left(\frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z})}{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\right)\right] + \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log\left(\frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right)\right].
\end{aligned}$$

Definimos **Evidence Lower Bound (ELBO)**

$$\begin{aligned}
\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) &:= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log\left(\frac{p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z})}{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\right)\right] \\
&= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{z}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\right]
\end{aligned}$$

y la **divergencia de Kullback-Leibler (KL)** entre $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$ y $p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ como

$$\begin{aligned}
D_{\mathrm{KL}}(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})) &:= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log\left(\frac{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}{p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})}\right)\right] \\
&= \mathbb{E}_{\boldsymbol{z}\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) - \log p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\right].
\end{aligned}$$

Nota 1.6.6

El término $\mathbb{E}_{z\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}[\log p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})]$ en $\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x})$ se conoce como **error de reconstrucción**.

De esta manera,

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) - D_{\mathrm{KL}}\big(q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \parallel p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big).$$

Nótese que

$$D_{\mathrm{KL}} \geq 0,$$

por lo que

$$\log p_{\boldsymbol{\theta}}(\boldsymbol{x}) \geq \mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}).$$

Además, observe que la divergencia KL determina dos distancias:

1. La distancia entre la posterior real, $p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$, y su aproximación, $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$, por la definición que tiene.
2. El *gap* entre la verosimilitud marginal $\log p_{\boldsymbol{\theta}}(\boldsymbol{x})$ y la ELBO $\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}$. (Llamado **ajuste de la cota**.)

Es por esto que intentaremos maximizar $\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}$ con respecto a sus parámetros $\boldsymbol{\theta}, \boldsymbol{\phi}$, de modo que ambas distancias se hagan lo más pequeñas posibles, de manera aproxima.

Stochastic Gradient-Based Optimization for ELBO

La ELBO permite SGD sobre sus parámetros.

Dado un dataset $\mathcal{D} = \boldsymbol{x}^{(1:N)}$, la ELBO objetivo es

$$\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathcal{D}) \coloneqq \sum_{i=1}^{N} \mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}\big(\boldsymbol{x}^{(i)}\big).$$

El gradiente individual $\nabla_{\boldsymbol{\theta},\boldsymbol{\phi}}\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}\big(\boldsymbol{x}^{(i)}\big)$ es en general intratable, pero existen buenos estimados insesgados $\tilde{\nabla}_{\boldsymbol{\theta},\boldsymbol{\phi}}\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}\big(\boldsymbol{x}^{(i)}\big)$.

Respecto a $\boldsymbol{\theta}$, es directo:

$$\begin{aligned}
\nabla_{\boldsymbol{\theta}}\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) &= \nabla_{\boldsymbol{\theta}}\mathbb{E}_{z\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\big[\log p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\big] \\
&= \nabla_{\boldsymbol{\theta}} \int q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\big[\log p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\big]\,\mathrm{d}\boldsymbol{z} \\
&\overset{\text{Leibniz}}{\longrightarrow} \quad = \int \nabla_{\boldsymbol{\theta}}\big\{q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\big[\log p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\big]\big\}\,\mathrm{d}\boldsymbol{z} \\
&= \int q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) \cdot \nabla_{\boldsymbol{\theta}}\big[\log p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\big]\,\mathrm{d}\boldsymbol{z} \\
&= \mathbb{E}_{z\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\big[\nabla_{\boldsymbol{\theta}}\big(\log p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\big)\big].
\end{aligned}$$

Tomando un $\boldsymbol{z} \sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$, se obtiene un estimador simple de Monte Carlo

$$\nabla_{\boldsymbol{\theta}}\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) \simeq \nabla_{\boldsymbol{\theta}}\big(\log p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\big) = \nabla_{\boldsymbol{\theta}} \log p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}).$$

Respecto a $\boldsymbol{\phi}$ es más difícil, pues $q_{\boldsymbol{\phi}}$ depende justamente de $\boldsymbol{\phi}$. Es decir,

$$\begin{aligned}\nabla_{\boldsymbol{\theta}}\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) &= \nabla_{\boldsymbol{\theta}}\mathbb{E}_{z\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\right] \\ &\neq \mathbb{E}_{z\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\nabla_{\boldsymbol{\theta}}\big(\log p_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\big)\right].\end{aligned}$$

Para esto, usaremos el *truco de reparametrización.*

Reparameterization Trick

Expresaremos $\boldsymbol{z} \sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$ como una transformación g diferenciable e invertible de otra va $\boldsymbol{\epsilon}$, independiente de $\boldsymbol{x}$ y de $\boldsymbol{\phi}$, y con densidad $p(\boldsymbol{\epsilon})$:

$$\boldsymbol{z} = g(\boldsymbol{\epsilon}, \boldsymbol{\phi}, \boldsymbol{x}).$$

Con esto, para cualquier función f,

$$\mathbb{E}_{z\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}[f(\boldsymbol{z})] = \int q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) f(\boldsymbol{z})\,\mathrm{d}\boldsymbol{z}.$$

Pero,

$$\mathrm{d}\boldsymbol{z} = \left|\det\left(\frac{\partial g}{\partial \boldsymbol{\epsilon}}\right)\right| \mathrm{d}\boldsymbol{\epsilon}$$

y

$$q(\boldsymbol{z}|\boldsymbol{x}) = \left|\det\left(\frac{\partial g}{\partial \boldsymbol{\epsilon}}\right)\right|^{-1} p(\boldsymbol{\epsilon}),$$

así que

$$q(\boldsymbol{z}|\boldsymbol{x})\,\mathrm{d}\boldsymbol{z} = p(\boldsymbol{\epsilon})\,\mathrm{d}\boldsymbol{\epsilon}.$$

Por lo tanto,

$$\begin{aligned}\mathbb{E}_{z\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}[f(\boldsymbol{z})] &= \int q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}) f(\boldsymbol{z})\,\mathrm{d}\boldsymbol{z} \\ &= \int f(g(\boldsymbol{\epsilon}, \boldsymbol{\phi}, \boldsymbol{x})) p(\boldsymbol{\epsilon})\,\mathrm{d}\boldsymbol{\epsilon} \\ &= \mathbb{E}_{\boldsymbol{\epsilon}\sim p(\boldsymbol{\epsilon})}[f(\boldsymbol{z})].\end{aligned}$$

Ahora se puede hacer

$$\begin{aligned}\nabla_{\boldsymbol{\phi}}\mathbb{E}_{z\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}[f(\boldsymbol{z})] &= \nabla_{\boldsymbol{\phi}}\mathbb{E}_{\boldsymbol{\epsilon}\sim p(\boldsymbol{\epsilon})}[f(\boldsymbol{z})] \\ &= \int \nabla_{\boldsymbol{\phi}}\{f(g(\boldsymbol{\epsilon}, \boldsymbol{\phi}, \boldsymbol{x})) p(\boldsymbol{\epsilon})\}\,\mathrm{d}\boldsymbol{\epsilon} \\ &= \int p(\boldsymbol{\epsilon}) \nabla_{\boldsymbol{\phi}} f(g(\boldsymbol{\epsilon}, \boldsymbol{\phi}, \boldsymbol{x}))\,\mathrm{d}\boldsymbol{\epsilon} \\ &= \mathbb{E}_{\boldsymbol{\epsilon}\sim p(\boldsymbol{\epsilon})}\left[\nabla_{\boldsymbol{\phi}} f(\boldsymbol{z})\right].\end{aligned}$$

Finalmente, tenemos un estimador simple de Monte Carlo

$$\nabla_{\boldsymbol{\phi}}\mathbb{E}_{z\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}[f(\boldsymbol{z})] \simeq \nabla_{\boldsymbol{\phi}} f(\boldsymbol{z}).$$

De esta manera, reparametrizamos al aleatoriedad de $\boldsymbol{z}$ y ahora podremos hacer *backpropagation* por $\boldsymbol{z}$ y calcular los gradientes $\nabla_{\boldsymbol{\phi}} f(\boldsymbol{z})$.

Nota 1.6.7

En este capítulo se enseña el *reparametrization trick* para una f cualquiera. En la sección que viene se usa una f del problema.

Gradiente de ELBO

Con el *reparametrization trick*, podemos escribir

$$\begin{aligned}\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) &= \mathbb{E}_{z\sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{z}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\right] \\ &= \mathbb{E}_{\boldsymbol{\epsilon}\sim p(\boldsymbol{\epsilon})}\left[\log p_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{z}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})\right].\end{aligned}$$

Luego, podemos obtener un estimador simple de Monte Carlo, $\tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x})$, de $\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x})$ para cada $\boldsymbol{x} \in \mathcal{D}$ (o $\mathcal{M}$):

$$\begin{aligned}\boldsymbol{\epsilon} &\sim p(\boldsymbol{\epsilon}) \\ \boldsymbol{z} &= g(\boldsymbol{\phi},\boldsymbol{x},\boldsymbol{\epsilon}) \\ \tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x}) &= \log p_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{z}) - \log q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x}).\end{aligned}$$

Finalmente, para optimizar el ELBO se ocupan los gradientes $\nabla_{\boldsymbol{\theta},\boldsymbol{\phi}}\tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\boldsymbol{x})$.

Algoritmo Auto-Encoding Variational Bayes (AEVB)

Por último, el algoritmo estocástico para optimizar el ELBO es

Algoritmo 1.6.8 AEVB

Data:

- $\mathcal{D}$: Dataset.
- $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$: Modelo de inferencia.
- $p_{\boldsymbol{\theta}}(\boldsymbol{x},\boldsymbol{z})$: Modelo generativo.

Resultado:

- $\boldsymbol{\theta},\boldsymbol{\phi}$: Parámetros aprendidos

1. $(\boldsymbol{\theta},\boldsymbol{\phi}) \leftarrow$ Inicializar parámetros.
2. **while** SGD *no ha convergido* **do**:
3. | $\mathcal{M} \sim \mathcal{D}$ (extraer un minibatch random).
4. | $\boldsymbol{\epsilon} \sim p(\boldsymbol{\epsilon})$ (extraer sonido random para cada punto en $\mathcal{M}$).
5. | Calcular $\tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathcal{M},\boldsymbol{\epsilon})$ y sus gradientes $\nabla_{\boldsymbol{\theta},\boldsymbol{\phi}}\tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathcal{M},\boldsymbol{\epsilon})$.
6. | Actualizar $\boldsymbol{\theta},\boldsymbol{\phi}$ usando un optimizador SGD.

Notas Adicionales

Ya sabemos que denotamos con $q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$ el encoder (o distribución posterior), con $p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z})$ el decoder y $p_{\boldsymbol{\theta}}(\boldsymbol{z})$ para el prior.

Una vez optimizado el ELBO con el algoritmo, ya tenemos $\boldsymbol{\theta},\boldsymbol{\phi}$ «óptimos», por lo que el encoder y el decoder ya están listos para ser usados. Con ellos, la *generación de datos nuevos* funciona del siguiente modo:

1. Samplee $\boldsymbol{z}' \sim p_{\boldsymbol{\theta}}(\boldsymbol{z})$ (prior).
2. Genere $\boldsymbol{x} \sim p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}')$ (decoder).

Nota 1.6.9

> Generar datos nuevos podría usarse, por ejemplo, para aumentar conjuntos pequeños de datos, o crear imágenes, sonidos, videos, etc.

Pero también se podría querer *reconstruir un dato*. Para ello, dado un dato $\boldsymbol{x}$:

1. Samplee $\boldsymbol{z}' \sim q_{\boldsymbol{\phi}}(\boldsymbol{z}|\boldsymbol{x})$ (encoder).
2. Genere $\boldsymbol{x}' \sim p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}')$ (decoder).

Nota 1.6.10

> Reconstruir un dato permite, por ejemplo, comprimir datos (guardando solamente $\boldsymbol{z}'$), eliminar ruido (guardando $\boldsymbol{x}'$ en vez de $\boldsymbol{x}$) o comprobar anomalías en el modelo (comparando $\boldsymbol{x}'$ con $\boldsymbol{x}$).

Glosario

Por hacer en el futuro

- **Inferencia Variacional:**
- **Estimador simple de Monte-Carlo:** es una manera de estimar, de manera insesgada, la esperanza de alguna cantidad aproximándola con el promedio de muestras aleatorias:

$$\mathbb{E}_{z\sim q_\phi(z)}[f(z)] \simeq \frac{1}{L}\sum_{k=1}^{L} f\big(z^{(l)}\big), \qquad \text{donde cada } z^{(l)} \sim q_\phi(z).$$

II Representaciones Latentes

2.1. Neural Discrete Representation

Nota 2.1.1 Fuente

> Contenido estudiado de *«Neural Discrete Representation Learning»* de A. van den Oord et al. (2017). [arXiv:1711.00937]

VQ-VAE es una familia de modelos que combina VAEs con cuantización de vectores (VQ) para obtener una representación latente discreta.

El paper busca aprender representaciones discretas útiles. Menciona que, en teoría, el mejor modelo (respecto a la logverosimilitud) es aquel que no usa variables latentes, sin embargo, estos modelos tienden a aprender ruido y pierden poder de cómputo con ello.

VQ-VAE

En los VAEs se suele tener que la distribución posterior $q(z|x)$ y la prior $p(z)$ son normales, por lo que se realiza el truco de reparametrización gaussiano. En el caso de VQ-VAE, estos espacios tienen *distribuciones categóricas*.

Variables Latentes Discretas

Definiremos nuestro espacio latente. Suponga que tendrá $K \in \mathbb{N}$ categorías o características importantes (estas irán creciendo con el entrenamiento). Para cada categoría $i \in \{1, ..., K\}$ definimos un **vector de *embedding*** $e_i \in \mathbb{R}^D$. Con ellos se define el **espacio latente de *embedding*** $e \in \mathbb{R}^{K \times D}$:

$$e = \begin{bmatrix} | & & | \\ e_1 & \cdots & e_K \\ | & & | \end{bmatrix}.$$

Seguidamente, describimos el pipeline general. Sea x un input. El modelo tomará dicho input y lo pasará por medio del encoder, produciendo $z_e(x)$. Aquí, definimos la **distribución categórica posterior** $q(z|x)$ como

$$q(z = k|x) = \begin{cases} 1 & \text{para } k = \arg\min_j \left\| z_e(x) - e_j \right\|_2 \\ 0 & \text{e.o.c.} \end{cases}$$

Por tanto, para un x particular, sólo existirá un $k \in \{1, ..., K\}$ tal que $q(z = k|x) = 1$.

A continuación, se le ingresa al decoder el vector de *embedding* e_k correspondiente:

$$z_q(x) = e_k.$$

Nota 2.1.2

- El modelo se puede pensar como VAE acotando $\log p(x)$ con la ELBO.
- La distribución $q(z = k|x)$ es determinista (dados los vectores e_i y el x).
- Definiendo una prior $p(z)$ uniforme, se obtiene la divergencia KL constante e igual a $\log K$.

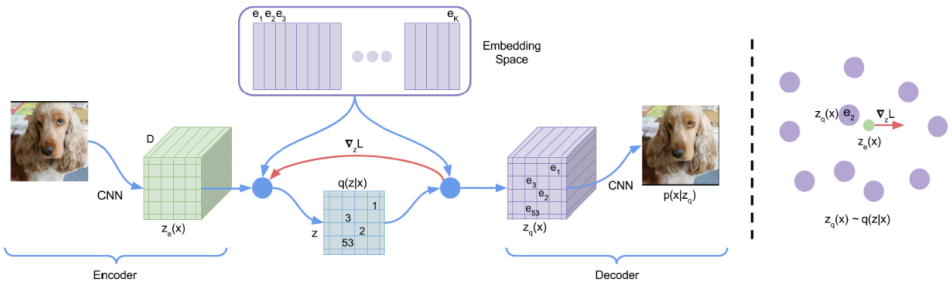


Figura 2 — A la izquierda, el pipeline de VQ-VAE. A la derecha, una representación del espacio de *embedding*

Aprendizaje

Se define la **función total de pérdida**

$$L = \log p\big(x|z_q(x)\big) + \|\text{sg}[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - \text{sg}[e]\|_2^2.$$

Descomposición:

- $\log p\big(x|z_q(x)\big)$: es la **pérdida de reconstrucción**. Permite optimizar el encoder y el decoder. Los *embeddings* e_i no reciben información de aquí.
- $\|\text{sg}[z_e(x)] - e\|_2^2$: es el uso de **Vector Quantization (VQ)**. Se usa un algoritmo de aprendizaje de diccionario que mueve los vectores e_i hacia las salidas del encoder $z_e(x)$. La función sg corresponde al **operador *stopgradient***, que se define como una identidad en tiempo computacional hacia adelante (?) y tiene derivadas parciales nulas.
- $\beta\|z_e(x) - \text{sg}[e]\|_2^2$: es la **pérdida de compromiso**. Penaliza la creación ilimitada de vectores de *embedding*, esto es, regula el crecimiento de $K \in \mathbb{N}$.

Se ajustarán algunos pesos por medio de *backpropagation*. Sin embargo, para pasar del decoder al encoder no hay un gradiente real (por la operación $k = \arg\min(\cdots)$). Aproximaremos el gradiente simplemente copiando el gradiente del input del decoder, $z_q(x)$, a la salida del encoder, $z_e(x)$.

Además, el algoritmo parece ser robusto a valores de $\beta \in [0.1, 2.0]$.

La logverosimilitud del modelo completo se puede evaluar como

$$\log p(x) = \log \sum_k p(x|z_k)p(z_k)$$

y se puede aproximar como $\log p\big(x|z_q(x)\big)p(z_k(x))$.

Prior y Distribuciones Autorregresivas

Diremos que una distribución es **autorregresiva** si la densidad en un punto/vector se puede representar como un producto condicionado a las variables/entradas anteriores. Esto es, para un $\boldsymbol{x} = (x_1, ..., x_T)$,

$$p(\boldsymbol{x}) = \prod_{t=1}^{T} p(x_t|x_1, ..., x_{t-1}).$$

Ellas permiten generación de datos secuenciados. PixelCNN y WaveNet basan su funcionamiento en este tipo de distribuciones.

Por último, la prior $p(z)$ es una distribución categórica que se asume constante y uniforme durante el entrenamiento. Al terminar el entrenamiento se ajusta a una distribución autorregresiva sobre z, lo que permite generar x por medio de sampleo ancestral. El **sampleo ancestral** (*ancestral sampling*) muestrea una variable condicionada a todas las anteriores. En nuestro contexto,

$$\begin{aligned} z_1 &\sim p(z_1) \\ z_2 &\sim p(z_2|z_1) \\ &\vdots \\ z_T &\sim p(z_T|z_1, ..., z_{T-1}). \end{aligned}$$

Así, se termina formando el vector $\boldsymbol{z} = (z_1, ..., z_T)$. Por lo tanto,

$$p(\boldsymbol{z}) = \prod_{t=1}^{T} p(z_t|z_1, ..., z_{t-1}).$$

Glosario

- **Embedding**: Transformación de un espacio categórico a uno continuo. Su función es aprender una representación de un espacio en otro. Pueden reducir, mantener o aumentar la dimensionalidad del espacio. La transformación se realiza mediante una matriz de pesos.
- **Codificación**: Similar al *embedding*, pero la transformación va de un espacio continuo a otro continuo.

2.2. Categorical Reparameterization with Gumbel-Softmax

Nota 2.2.1 Fuente

> Contenido estudiado de *«Categorical Reparameterization with Gumbel-Softmax»* de E. Jang et al. (2016). [arXiv:1611.01144]

El paper presenta un estimador de gradiente eficiente que reemplaza las muestras no diferenciables a partir de una nueva distribución Gumbel-Softmax, la cual puede ser *annealed* en una distribución categórica.

La Distribución Gumbel-Softmax

La distribución Gumbel-Softmax es una distribución continua sobre un símplex que puede aproximar muestras de una distribución categórica. Un **símplex** de dimensión $k-1$, Δ^{k-1}, es un espacio (NO vectorial) que contiene vectores k-dimensionales con entradas no negativas y cuya suma es 1, es decir,

$$\Delta^{k-1} := \left\{ (x_1, ..., x_k) \in \mathbb{R}^k : x_i \geq 0 \ \text{ y } \sum_{i=1}^{k} x_i = 1 \right\}.$$

Note que cada elemento $x \in \Delta^{k-1}$ induce una distribución de probabilidad.

Sea z una variable categórica con probabilidades de clase $\pi_1, ..., \pi_k$ (es decir, z tiene probabilidad π_i de pertenecer a la clase $i \in [k]$).

Nota 2.2.2

> El paper asume que las muestras categóricas son vectores one-hot en Δ^{k-1}.

Con esto, tenemos $z = e_i$, para algún $i \in [k]$ y donde e_i representa al i-ésimo vector canónico. Por lo tanto,

$$\mathbb{E}_p[z] = \sum_{j=1}^{k} e_j P(z = e_j) = \sum_{j=1}^{k} e_j p(e_j) = \sum_{j=1}^{k} e_j \pi_j = [\pi_1, ..., \pi_k].$$

El **Gumbel-Softmax trick** es un método para obtener muestras z de una distribución con probabilidades de clase π:

$$z = \text{one_hot}\Big(\arg\max_i [g_i + \log \pi_i]\Big),$$

donde $g_1, ..., g_k$ son muestras i.i.d. de una Gumbel$(0, 1)$ (una muestra de esta Gumbel$(0, 1)$ se obtiene generando una muestra $u \sim$ Uniforme$(0, 1)$ y luego aplicándole $-\log(-\log(u))$).

Para solucionar la falta de diferenciabilidad del arg max en el Gumbel-Softmax trick, usaremos la función softmax, que es continua y diferenciable. La **función softmax** $\sigma : \mathbb{R}^k \to [0, 1]^k$ sobre un vector $z = (z_1, ..., z_k)$ se define como

$$(\sigma(z))_i = \frac{e^{z_i}}{\sum_{J=1}^{k} e^{z_j}} \qquad \text{para todo } i = 1, ..., k.$$

[2] Sabiendo esto, podremos generar vectores $y = (y_1, ..., y_k) \in \mathbb{R}^k$ tales que

$$y_i = \frac{\exp((g_i + \log \pi_i)/\tau)}{\sum_{j=1}^{k} \exp\big((g_j + \log \pi_j)/\tau\big)},$$

donde $\tau \in \mathbb{R}^+$ es un parámetro denominado **temperatura**. Para estos vectores y (generados por este proceso de muestrear de una Gumbel$(0, 1)$), podemos calcular la densidad, llamada **densidad de la distribución Gumbel-Softmax**,

$$p_{\pi,\tau}(y_1, ..., y_k) = \Gamma(k)\tau^{k-1}\left(\sum_{i=1}^{k} \pi_i/y_i^{\tau}\right)^{-k} \prod_{i=1}^{k}(\pi_i/y_i^{\tau+1}).$$

Nota 2.2.3

> Esta distribución fue descubierta independientemente por Maddison *et al.* y a veces se le conoce como **Concrete Distribution**.

Conforme $\tau \to 0^+$, las muestras de la distribución se convierten en one-hot y la distribución se convierte en la distribución categórica $p(z)$.

El Estimador Gumbel-Softmax

La distribución Gumbel-Softmax es suave para $\tau > 0$, por tanto, tiene un gradiente bien definido

$$\frac{\partial y}{\partial \pi},$$

con respecto a los parámetros π. Por tanto, si reemplazamos muestras de la distribución categórica $p(z)$ por muestras de la Gumbel-Softmax $p_{\pi,\tau}$, podemos usar *backpropagation*. Este proceso de reemplazar muestras no diferenciables con una aproximación diferenciable como el **estimador Gumbel-Softmax**.

En el aprendizaje hay un *tradeoff* entre valores de τ pequeños, con muestras cercanas a muestras one-hot, pero valores de varianza grandes, contra muestras más suaves, pero con menor varianza. En la práctica se recomienda partir con alta temperatura e ir haciendo cada vez más pequeño el $\tau > 0$ (pudiendo seguir una gran variedad de rutinas de actualización de su valor).

Estimador Gumbel-Softmax Straight-Through

Las relajaciones continuas de los vectores one-hot (i.e. de las muestras) pueden ser adecuados para ciertos problemas. Sin embargo, habrá situaciones donde esto no sea posible porque necesariamente se deberá muestrar de un espacio discreto (e.g. un espacio discreto de acciones en RL). En esos caso, se discretiza y usando arg max (esto es, se ocupa la definición que dimos de z con el Gumbel-Softmax trick), pero se usa la aproximación continua en el *backward pass*, de modo que se aproxime el gradiente discreto por su relajación continua,

$$\nabla_\theta z \approx \nabla_\theta y.$$

[2] Una propiedad muy útil es que conserva el orden relativo $(\leq)$ de las entradas, así que el máximo de z se encuentra en la misma entrada que la misma entrada de $\sigma(z)$.

Este procedimiento lo denominamos **Estimador Gumbel Straight-Through (ST)**. Este estimador permite que las muestras sean dispersas (con muchos ceros, ergo «cercanas» a ser one-hot) incluso con temperaturas τ grandes.

Resultados Experimentales

En los experimentos se prueban los dos estimadores: el Gumbel-Softmax estándar (que siempre aproxima el arg max con la función softmax) y el ST Gumbel-Softmax (que sólo aproxima arg max en el *backward pass*).

Modelos de Predicción de Salidas Estructuradas

En redes binarias estocásticas, ST Gumbel-Softmax obtiene resultados similares a otros estimadores en variables Bernoulli y los supera en variables categóricas.

El estimador Gumbel-Softmax estándar supera al resto de estimadores tanto en variables Bernoulli como en variables categóricas.

Modelos Generativos (VAEs)

El estimador ST Gumbel-Softmax supera a otros estimadores en el uso de variables categóricas.

El estimador Gumbel-Softmax supera a todos los demás estimadores tanto en variables de Bernoulli como en variables categóricas.

Clasificación Semisupervisada Generativa

El **aprendizaje semisupervisado** consiste en aprender de un conjunto que mezcla datos etiquetados como datos no etiquetados.

Para una sola muestra, Gumbel-Softmax mantiene un rendimiento generativo y de clasificación equivalente al método de marginalización. El **método de marginalización** consiste en calcular la probabilidad sobre todas las clases posibles de una variable categórica y, permitiendo entrenar también con datos no etiquetados, donde y no fue observada.

2.3. Posterior Collapse

Los VAEs son redes neuronales generativas que consisten en:

- Un **prior** $p_\theta(\boldsymbol{z})$.
- Un **encoder** $q_\phi(\boldsymbol{z}|\boldsymbol{x})$.
- Un **decoder** $p_\theta(\boldsymbol{x}|\boldsymbol{z})$.

En 1.6. An Introduction to Variational Autoencoders probamos que $\mathbb{E}(\log p_{\boldsymbol{\theta}}(\boldsymbol{x}))$ se puede expresar como una suma de la ELBO y la divergencia KL. Esta esperanza indica qué tan bien se reconstruye un $\boldsymbol{x}$ es particular y la divergencia KL mide qué tan distintas son dos distribuciones, en particular, las distribuciones del prior y del encoder. Se ha propuesto muchas veces que el *posterior collapse* surge por este último término.

Posterior Collapse as a Phase Transition in Variational Autoencoders

Nota 2.3.1 Fuente

> Contenido estudiado de *«Posterior Collapse as a Phase Transition in Variational Autoencoders»* de Z. Li et al. (2025). [arXiv:2510.01621]

Aquí se estudia el *posterior collapse* como un problema de física estadística, proponiendo que existe un punto crítico (una *transición de fase*) en los VAEs, debido a la estructura y al dataset utilizado, que marca el cambio a un *posterior collapse*.

El **colapso posterior** (*posterior collapse*) ocurre cuando la posterior $q_\phi(\boldsymbol{z}|\boldsymbol{x})$ ignora un subconjunto del espacio latente. Es decir, para vectores $\boldsymbol{z}$ en un subconjunto Z,

$$q_\phi(\boldsymbol{z}|\boldsymbol{x}) = p_\theta(\boldsymbol{z}).$$

Se ha propuesto anteriormente que esto surge de la divergencia KL, por lo que se han propuesto ajustes como el β-VAE, que pondera la divergencia KL en la maximización de la logverosimilitud. Sin embargo, otros enfoques analíticos (en especial los linealizados) han mostrado que PARTE (?) de las causas pueden venir de otros lados. Esto ha llevado a los autores a tomar un enfoque analítico, pero fuera del caso exclusivamente lineal.

El paper estudia la maximización del ELBO por medio de cálculo de variaciones (y multiplicadores de Lagrange) imponiendo ciertas restricciones sobre $p_\theta(\boldsymbol{x}|\boldsymbol{z})$ y $q_\phi(\boldsymbol{z}|\boldsymbol{x})$. Con esto, obtienen que los puntos óptimos son aquellos donde

$$p_\theta(\boldsymbol{x}|\boldsymbol{z}) = p(\boldsymbol{x}|\boldsymbol{z}) \quad \text{y} \quad q_\phi(\boldsymbol{z}|\boldsymbol{x}) = p(\boldsymbol{z}|\boldsymbol{x}).$$

Es decir, donde el decoder y el encoder parametrizados son iguales a las distribuciones que aproximan.

Se hace énfasis en que hay múltiples valores extremos y que, un punto especial es el *posterior collapse*:

$$p_\theta(\boldsymbol{x}|\boldsymbol{z}) = p_{\text{data}}(\boldsymbol{x}) \quad \text{y} \quad q_\phi(\boldsymbol{z}|\boldsymbol{x}) = p_{\text{lat}}(\boldsymbol{z}).$$

Aquí, ocurren dos colapsos:

- El encoder ignora a los vectores $\boldsymbol{x}$.

- La única solución del modelo es aprender la distribución marginal (el promedio) sobre el conjunto de entrenamiento, esto es, la distribución real de los datos, lo que fuerza a que la reconstrucción no tenga nada de información real respecto al dato original.

El paper a continuación le da una forma particular a $p_\theta(\boldsymbol{x}|\boldsymbol{z})$, $q_\phi(\boldsymbol{z}|\boldsymbol{x})$ y a $p_{\text{lat}}(\boldsymbol{z})$, haciéndolas normales. (Esto se conoce como **deep Gaussian VAEs**.) Resolviendo, se obtiene un punto extremo trivial,

$$\begin{aligned} f_i(\boldsymbol{z}) &= E_{p_{\text{data}}(x)}[x_i] \\ \mu_j(\boldsymbol{x}) &= 0 \\ \sigma_j(\boldsymbol{x}) &= 1, \end{aligned}$$

que corresponde al colapso posterior. Luego, se obtiene el valor extremo

$$\mathcal{L}^{G*}_{\text{VAE}} = -\frac{N}{2}\ln 2\pi\sigma'^2 - \frac{1}{2\sigma'^2}\ \text{Tr}(\boldsymbol{\Sigma}),$$

donde $\boldsymbol{\Sigma}$ es matriz de covarianza de los datos $\boldsymbol{x}$ y σ'^2 un hiperparámetro (la varianza de $p_\theta(\boldsymbol{x}|\boldsymbol{z})$).

En el paper se demuestra que el criterio

$$\sigma'^2 > \max[\xi_1^2, ..., \xi_N^2],$$

donde los ξ_i^2 son los valores propios de la matriz de covarianza de los datos, aproxima si ocurre o no el colapso posterior. Esto se puede interpretar como que, si los datos tienen demasiado ruido, el modelo termina decidiendo ignorarlos y simplemente reconstruir la distribución real.

Notas sobre Notación

Distribuciones Teóricas (Verdaderas)

- $p_{\text{data}}(\boldsymbol{x})$: La distribución marginal real de los datos. Describe la probabilidad de observar los datos en el mundo real y es matemáticamente intratable.
- $p(\boldsymbol{z}|\boldsymbol{x})$: La distribución posterior verdadera. Describe los factores latentes z que generaron un dato específico x. Es intratable y representa el punto óptimo analítico que el modelo intenta alcanzar (Eq. 7).
- $p(\boldsymbol{x}|\boldsymbol{z})$: La distribución condicional verdadera de los datos. Describe cómo se generan los datos a partir de factores latentes exactos. Es el punto óptimo analítico exigido para lograr una reconstrucción perfecta (Eq. 6).

Distribuciones Parametrizadas (El Modelo)

- $q_\phi(\boldsymbol{z}|\boldsymbol{x})$: La distribución posterior aproximada, implementada mediante el **encoder**. Utiliza parámetros ϕ (pesos de una red neuronal) para aproximarse a la intratable $p(\boldsymbol{z}|\boldsymbol{x})$.
- $p_\theta(\boldsymbol{x}|\boldsymbol{z})$: La distribución de reconstrucción, implementada mediante el **decoder**. Utiliza parámetros θ para generar o reconstruir datos $\boldsymbol{x}$ a partir de los latentes $\boldsymbol{z}$.

Distribuciones Previas (Priors)

- $p_{\text{lat}}(\boldsymbol{z})$: La distribución a priori estática sobre las variables latentes. Establece la estructura matemática deseada del espacio latente antes de observar cualquier dato (típicamente Gaussiana estándar).

- $p_\theta(\boldsymbol{z})$: Una distribución a priori alternativa donde sus parámetros θ se optimizan durante el entrenamiento del modelo (no utilizada en las formulaciones del documento analizado).

Posterior Collapse and Latent Variable Non-identifiability

Nota 2.3.2 Fuente

> Contenido estudiado de *«Posterior Collapse and Latent Variable Non-identifiability»* de Y. Wang et al. (2021). [arXiv:2301.00537]

El paper demuestra que un VAE sufre de *posterior collapse* si, y sólo si, la variable latente del modelo es no-identificable. Además, proponen un nuevo tipo de VAE, el *latent-identifiable VAE*, el cual resuelve el colapso posterior sin sacrificar fidelidad (información relevante) a los datos.

Definición 2.3.3 Colapso posterior

> Dado un modelo de probabilidad $p(\boldsymbol{x}, \boldsymbol{z}; \theta)$, un valor de parámetro $\theta = \hat{\theta}$ y un dataset $\boldsymbol{x} = (x_1, ..., x_n)$, la posterior de las variables latentes $\boldsymbol{z}$ **colapsa** si
>
> $$p\left(\boldsymbol{z}|\boldsymbol{x}; \hat{\theta}\right) = p(\boldsymbol{z}).$$

El colapso posterior no permite que la variable latente dé un resumen significativo del dataset.

Definición 2.3.4 Variable latente no-identificable

> Dada una función de verosimilitud $p(\boldsymbol{x}|\boldsymbol{z}; \theta)$, un valor de parámetro $\theta = \hat{\theta}$ y un dataset $\boldsymbol{x} = (x_1, ..., x_n)$, la variable latente $\boldsymbol{z}$ es **no-identificable** si
>
> $$p\left(\boldsymbol{x}|\boldsymbol{z} = \tilde{\boldsymbol{z}}'; \hat{\theta}\right) = p\left(\boldsymbol{x}|\boldsymbol{z} = \tilde{\boldsymbol{z}}; \hat{\theta}\right) \quad \text{para cualesquiera } \tilde{z}', \tilde{z} \in \mathcal{Z}.$$

Como consecuencia de la no-identificabilidad, tenemos que

$$p\left(\boldsymbol{x}|\boldsymbol{z} = \tilde{\boldsymbol{z}}; \hat{\theta}\right) = p\left(\boldsymbol{x}; \hat{\theta}\right) \quad \text{para todo } \tilde{z} \in \mathcal{Z}.$$

Para asegurar que la variable latente será identificable, basta con asegurar que la verosimilitud $p(\boldsymbol{x}|\boldsymbol{z}, \theta)$ es inyectiva en $\boldsymbol{z}$, para todo θ y $\boldsymbol{x}$ fijo.

Nota 2.3.5

> La identificabilidad depende del dataset $\boldsymbol{x}$, por lo que una variable latente podría ser identificable en un modelo, pero no en otro.

Teorema 2.3.6 Caracterización del colapso posterior

> Considere un modelo de probabilidad $p(\boldsymbol{x}, \boldsymbol{z}; \theta)$, un dataset $\boldsymbol{x}$ y un valor de parámetro $\theta = \hat{\theta}$. Luego, las variables latentes locales $\boldsymbol{z}$ son no-identificables en $\hat{\theta}$ si, y sólo si, la posterior de la variable latente $\boldsymbol{z}$ colapsa.

Demostración

($\Longrightarrow$) Por regla de Bayes,

$$p\left(\boldsymbol{z}|\boldsymbol{x};\hat{\theta}\right) \propto p(\boldsymbol{z})p\left(\boldsymbol{x}|\boldsymbol{z};\hat{\theta}\right) = p(\boldsymbol{z})p\left(\boldsymbol{x}|\hat{\theta}\right) \propto p(\boldsymbol{z}),$$

donde la igualdad se obtiene de la no-identificabilidad.

($\Longleftarrow$) El colapso implica, por Bayes,

$$p(\boldsymbol{z}) = p\left(\boldsymbol{z}|\boldsymbol{x};\hat{\theta}\right) \propto p(\boldsymbol{z})p\left(\boldsymbol{x}|\boldsymbol{z};\hat{\theta}\right),$$

por lo que $p\left(\boldsymbol{x}|\boldsymbol{z};\hat{\theta}\right)$ debe ser constante en $\boldsymbol{z}$ y, por tanto, hay no-identificabilidad. ■

Es de notar que la naturaleza de la caracterización entregada puede ser aplicada a modelos más allá de las VAEs, por ejemplo, en el Probabilistic Principal Component Analysis (PPCA).

A continuación, el paper presenta algunos ejemplos de variables no-identificables y colapso posterior (lo omití por honor al tiempo).

Luego, se presenta el **latent-identifiable VAE (LIDVAE)** via Brenier maps, un modelo VAE que garantiza ser identificable y, por tanto, no colapsar. Aquí, se construyen verosimilitudes usando mapeos de Brenir y una aplicación matricial inyectiva, resultando en un mapeo inyectivo.

Resultados

Al aplicar el LIDVAE a texto e imágenes, se evita el posterior collapse

¿Cómo se compara este paper con el anterior?

Los papers son distintos en objetivos:

- «Posterior Collapse as a Phase Transition in Variational Autoencoders» busca obtener un criterio para predecir el colapso posterior basándose en métodos de física estadística y, en particular, obtiene uno especial para el caso donde las distribuciones son normales.
- «Posterior Collapse and Latent Variable Non-identifiability» busca una caracterización del colapso posterior, probando una equivalencia con la no-identificabilidad. Además, muestra que esta caracterización es más general a sólo los VAEs y propone un modelo que evita la no-identificabilidad con mapeos de Brenier.

VQ-VAE y Posterior Collapse

El VQ-VAE se distingue en dos aspectos del VAE original:

1. Produce códigos discretos, no continuos (es decir, $\boldsymbol{z}$ es un vector continuo).
2. Aprende durante el entrenamiento.

Como recordatorio, el VQ-VAE parte desde un **codebook** de K vectores aprendibles $e_1, ..., e_K \in \mathbb{R}^D$, toma un ejemplo $\boldsymbol{x}$ y el encoder produce un vector $z_e(\boldsymbol{x})$ que se reemplazará por el vector más cercano del codebook, formando $z_q(\boldsymbol{x})$ (así que la posterior $q_\phi(\boldsymbol{z}|\boldsymbol{x})$ es determinista). Finalmente, el $z_q(\boldsymbol{x})$ se pasa al decodificador $p_\theta(\boldsymbol{x}|\boldsymbol{z})$.

La ventaja del VQ-VAE es que, al definir uniforme la distribución sobre los $\boldsymbol{z}$, la divergencia KL se hace constante (igual a $\log K$) respecto a los parámetros θ, ϕ, por lo que **ya no se empuja** a $q_\phi(\boldsymbol{z}|\boldsymbol{x})$ al prior, lo que **evita el colapso posterior**.

III Síntesis de Programas

3.1. DreamCoder

Nota 3.1.1 Fuente

> Contenido estudiado de *«DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning»* de K. Ellis et al. (2021). [arXiv:2006.08381]

Nota 3.1.2

> No confundir con **5.1. DreamerV3**.

DreamCoder es un modelo de síntesis de programas. Es decir, a partir de un conjunto de pares input-output, se busca generar un programa que permita realizar esos mismos cálculos y luego generalizar. El algoritmo, a grandes rasgos, itera creando código y luego factorizando el código en funciones que busca reutilizar.

Síntesis Inductiva de Programas

Dado un conjunto de tareas $\mathcal{X} = \{x_1, ..., x_n\}$, cada una especificada por un pequeño número de pares input-output de ejemplo, buscamos encontrar programas $\left\{\varrho_{x_i}\right\}$ que satisfacen todos los ejemplos para cada tarea, respectivamente.

El problema tiene tres retos esenciales:

1. Explosión combinatorial: el número de programas válidos sintácticamente crece exponencialmente.
2. Generalización, no interpolación: se busca que el programa sea correcto incluso con inputs fuera del training set.
3. Interpretabilidad y reuso: idealmente el programa debe ser interpretable por un humano y deseablemente reutilizable a futuro.

Fundación en el MDL

Definición 3.1.3

> Para una librería L y programas $\{\varrho_i\}$ que resuelven tareas en $\mathcal{X}$, el **joint description length** es
>
> $$\mathrm{DL}(L, \{\varrho\}, \mathcal{X}) := |L| + \sum_{x \in \mathcal{X}} |\varrho_x|_L .$$

El objetivo MDL aquí es encontrar $(L^*, \{\varrho_x^*\})$ que minimiza el joint description length sujeto a que ϱ_x resuelve $x \in \mathcal{X}$.

Después de un poco de desarrollo, se obtiene que el objetivo de DreamCoder corresponde justamente al del MDL. Sin embargo, el objetivo obtenido es intratable por la explosión combinatorial. Para ello, se introduce el **amortised recognition model** $Q_\theta(\varrho|x)$, una red neuronal que permite aproximar una distribución que era intratable. El rol de Q_θ es guiar la búsqueda de programas, prediciendo qué programas son más probables de resolver las tareas dadas.

Domain-Specific Languages (DSL)

Los Domain-Specific Languages permiten resolver el problema de la explosión combinatorial. Sin embargo, hasta antes de esta investigación, estos se daban desde un inicio. En este paper es el mismo modelo el que crea su DSL.

Wake-Sleep Algorithm

DreamCoder realiza inferencia bayesiana aproximada por medio un proceso iterativo que tiene tres fases:

- **Wake:** actualiza $\{\varrho_x\}$ manteniendo L y θ fijas. Busca programas usando Q_θ y L.
- **Sleep (Abstraction):** actualiza L, manteniendo $\{\varrho_x\}$ y θ fijas. Comprime los programas en L.
- **Sleep (Dreaming):** actualiza θ manteniendo $\{\varrho_x\}$ y L fijas. Entrena Q_θ en replays y fantasías.

Wake phase

Se busca para cada tarea $x \in \mathcal{X}$ el programa con mayor puntuación, dada por el modelo de reconocimiento Q_θ y la prior P:

$$\text{score}(\varrho, x) = \log Q_\theta(\varrho|x) + \underbrace{\log P(\varrho, L)}_{-|\varrho|_L}.$$

La implementación usa beam-first-search. Los programas que no son correctos para todos los ejemplos de la tarea son descartados. Además, se tiene un presupuesto de tiempo de entrenamiento. Pasado el presupuesto, se detiene la búsqueda. (Si no se encontró un programa, la tarea simplemente permanece no resuelta.)

Sleep Abstraction Phase

Se busca una librería L^* que introduce nuevo vocabulario sólo si extender L^* tiene mayor beneficio en compresión que el perjuicio en longitud. Para esto se hace

$$L^* = \arg\max_L \left[\log P(L) + \sum_{x \in \mathcal{X}} \log P(\varrho_x^*|L) \right],$$

donde $\log P(L) = -\beta \text{ size}(L)$ es un regularizador que permite aplicar el MDL.

Sleep Dreaming Phase

Aquí se mejora la predicción de Q_θ sobre $P(\varrho|x, L)$. El objetivo de entrenamiento es

$$\theta^* = \arg\max_\theta E_{(\varrho,x)\sim P_{\text{train}}}[\log Q_\theta(\varrho|x)],$$

donde P_{train} es una mezcla de:

- Replays: pares (x, ϱ_x^*). Permiten que Q_θ no olvide cómo resolver tareas que ya ha resuelto.
- Fantasías: pares (x, ϱ), donde $\varrho \sim P(\varrho|L)$ y x se induce ejecutando ϱ en inputs random. Ellas permiten tener un suministro ilimitado de datos sintéticos para el entrenamiento y permiten que Q_θ generalice más allá del pequeño training set.

Nota 3.1.4 Conexión con DreamerV3

El rol de las fantasías es similar al del modelo de mundo que usas DreamerV3. Ambos permiten experimentar ilimitadamente y ensayar con datos que no ha obtenido de la fuente original, basándose en la experiencia pasada.

Los dos papers siguientes intentan obtener lo mejor de dos mundos de síntesis de programas, el enfoque simbólico (o inductivo) y el enfoque neuronal (o transductivo).

3.2. Searching Latent Program Spaces

Nota 3.2.1 Fuente

> Contenido estudiado de *«Searching Latent Program Spaces»* de M. V. Macfarlane et al. (2024). [arXiv:2411.08706]

El paper presenta la arquitectura **Latent Program Network (LPN)** que busca sintetizar programas a partir de pocos ejemplos y ajustarlos en caso de que la primera intuición no alcance. Para ello, toma elementos de dos acercamientos a la resolución de ARC-AGI (que los autores defienden como una buena manera de medir la inteligencia por medir la capacidad de adaptarse a nuevas tareas):

- **Acercamiento inductivo** / *Síntesis de programas simbólica*: Se produce un programa explícito, pero el espacio de búsqueda es demasiado grande, por lo que requiere un lenguaje de dominio específico (DSL) diseñado a mano que limite las opciones, algo poco escalable a tareas del mundo real. LPN de aquí usa la capacidad de búsqueda durante el momento de prueba.
- **In-context/transductivo** / *Aprendizaje profundo*: Las redes neuronales son muy escalables, pero luego de entrenarse fijan sus pesos, por lo que suele ser necesario reentrenar y, con pocos datos, suelen overfittear. Además, tienden a ser inconsistentes en reproducir los ejemplos que se les dieron. LPN de aquí toma la predicción principal.

LPN representará los programas implícitos en un espacio continuo latente de programas y usará backpropagation en su fase de *fine-tunning* con una fracción del total de parámetros.

El paper propone realizar test-time training, que por lo general es muy costoso pues implica la actualización de millones de parámetros, en un *espacio latente continuo y compacto* que la propia red aprendió (en vez de hacerlo en el espacio de parámetros de la red neuronal o en el espacio de programas de un DSL).

Background

Se define el **espacio de programas** Y formulados a partir de un DSL. Cada tarea está definida por un **conjunto de especificación** X, donde cada **especificación** $X_m \in X$ se describe como un conjunto de pares input-output,

$$X_m = \{(x_1^m, y_1^m), ..., (x_n^m, y_n^m)\}.$$

Un programa $f \in Y$ resuelve la tarea asociada a X_m si

$$\forall j \in [n], \quad f\left(x_j^m\right) = y_j^m.$$

Denotaremos con F_m a la función verdadera que generó X_m, es decir, $F_m\left(x_j^m\right) = y_j^m$ para cada $j \in [n]$.

El problema de generalización se define a partir del conjunto

$$P_m = \{(x_1^m, y_1^m), ..., (x_n^m, y_n^m), x_{n+1}^m\},$$

donde se buscará predecir el output correspondiente de x_{n+1}^m.

El paper menciona que, de no limitar la complejidad de Kolmogorov del programa, podemos potencialmente encontrar un programa para cualquier especificación, aun si corresponde o no al programa original F_m. (¡Esto es una gran puerta al overfitting!)

Arquitectura LPN

El proceso de inferencia de LPN ocurre en tres etapas:

$$\text{Encoder} \longrightarrow \begin{array}{c}\text{Optimización en el} \\ \text{espacio latente}\end{array} \longrightarrow \text{Decoder}$$

El encoder generará una primera «intuición», la optimización en el espacio latente buscará refinar dicha intuición y el decoder ejecutará el programa refinado sobre el nuevo ejemplo.

El Espacio Latente de Programas

Un **espacio latente** es un espacio vectorial de dimensión reducida donde una red neuronal representa de forma comprimida y abstracta la información relevante de una entrada mucho más compleja.

Autoencoder Variacionales (VAE) y LPN

Un **autoencoder variacional (VAE)** es una red neuronal generativa que toma datos de entrada, los transforma a un espacio latente continuo según una distribución de probabilidad (lo que permitirá generar nuevos datos) y toma los puntos de dicho espacio para reconstruirlos o transformarlos en nuevos datos.

Más formalmente, un VAE tiene:

- Un **encoder** $q_\phi(z|x,y)$: dado un dato (x,y) de entrada, produce parámetros de media μ y varianza Σ de una distribución de probabilidad Normal multivariada sobre los posibles valores de un vector z en el espacio latente de programas. Además, el encoder procesa cada par por separado y luego los promedia, de modo que el LPN sea invariante a la *permutación de ejemplos*. ϕ corresponde a los pesos de la red neuronal del encoder.
- Un **decoder** $p_\theta(y|x,z)$: dado un programa latente z y un input nuevo x, reconstruye los datos de salida y. Se construye maximizando la verosimilitud de reconstruir bien los datos. θ corresponde a los pesos de la red neuronal del decoder.

El VAE permitirá modelar la ambigüedad de los muchos programas posibles con la distribución de probabilidad.

Nota 3.2.2

> Puedes aprender más sobre VAEs en 1.6. An Introduction to Variational Autoencoders.

La Optimización en el Espacio Latente

Tendremos

$$\underbrace{z \sim q_\phi(z \mid x, y)}_{\text{Encoder}} \qquad \underbrace{z' = f(p_\theta, z, x, y)}_{\text{Optimización latente}} \qquad \underbrace{\hat{y} \sim p_\theta(y \mid x, z)}_{\text{Decoder}}$$

El paso intermedio parte desde z, donde se busca un mejor programa z' que explique, bajo el actual decoder, de mejor manera los datos observados.

Dados datos $\{(x_i, y_i)\}_{i=1}^n$, el proceso de búsqueda $z' = f(p_\theta, z, x, y)$ consiste en encontrar un z' que satisfaga

$$z' \in \arg\max_z \sum_{i=1}^{n} \log p_\theta(y_i \mid x_i, z),$$

esto es, se busca el programa que más probabilidades tenga de producir resultados correctos bajo el decoder actual y los datos que se tienen.

Se proponen dos maneras de encontrar z':

- Por **muestreo**: se generan K programas z_k muestreados (de alguna distribución, ya sea $p(z)$ o la posteriori $q_\phi(z|x_i, y_i)$) y se elige el que tenga mayor log-verosimilitud. Su eficiencia decrece exponencialmente respecto a la dimensión del espacio latente.
- Por **Ascenso de Gradiente**: el decoder es una red neuronal diferenciable y, en particular, también su log-verosimilitud respecto a z. Se inicializa en z'_0 (promedio de todos los pares latentes sampleados del encoder) y se itera con

  $$\forall k \in [1, K], \quad z'_k = z'_{k-1} + \alpha \cdot \nabla_z \sum_{i=1}^{n} \log p_\theta(y_i \mid x_i, z)\Big|_{z=z'_{k-1}}.$$

  El ascenso de gradiente aquí se realiza en el espacio latente de programas. Como el espacio es de menor dimensión que el de los parámetros de la red, la optimización es mucho más barata.

Fase de Entrenamiento Offline

Aquí se trata cómo se obtienen los valores de ϕ (parámetros del encoder) y de θ (parámetros del decoder).

Durante el entrenamiento se utiliza la estrategia «**leave-one-out**» (LOO), donde, para reconstruir cada y_i:

1. Con cada $j \neq i$, se muestrea $z_j \sim q_\phi(z \mid x_j, y_j)$ con el encoder. La distribución es una $\mathcal{N}(\mu_j, \Sigma_j)$, cuyos parámetros produce el encoder.
2. Se agregan promediándolos: $\overline{z} = \frac{1}{n-1} \sum_{j \neq i} z_j$.
3. Se obtiene un z'_i con optimización en el espacio latente (e.g. con ascenso de gradiente [¡Por esto se diferencia $\mathcal{L}_{\text{total}}$ respecto a ϕ!]).
4. Se calcula la log-verosimilitud de predecir correctamente y_i dado x_i y z'_i con el decoder (θ fijo en este instante).

Además, la función objetivo a entrenar es

$$\mathcal{L}_{\text{total}}(\phi, \theta) = \mathcal{L}_{\text{rec}}(\phi, \theta) + \beta \mathcal{L}_{\text{KL}}(\phi)$$

se subdivide en:

- **Pérdida de reconstrucción** $\mathcal{L}_{\text{rec}}$:

  $$\mathcal{L}_{\text{rec}}(\phi, \theta) = \sum_{i=1}^{n} -\log p_\theta(y_i \mid x_i, z'_i)$$

  Mide qué tan mal predice el *decoder* las salidas correctas.

- **Pérdida de regularización $\mathcal{L}_{\mathrm{KL}}(\phi)$:**

$$\mathcal{L}_{\mathrm{KL}}(\phi) = \sum_{i=1}^{n} D_{\mathrm{KL}}\big(q_\phi(z \mid x_i, y_i) \parallel \mathrm{Normal}(0, I)\big)$$

  Mide qué tan lejos está la distribución que produjo el *encoder* de una distribución Normal (gracias a la **divergencia de Kullback-Leibler**), ayudando a que esta sea lo suficientemente suave para poder ser diferenciada.

- Factor β: Regula qué tanta importancia tiene un término por sobre otro. (Este parámetro hace que esta investigación se considere un β-VAE, aunque en los experimentos usan $\beta \approx 10^{-4} < 1$, y muchas veces en los β-VAE se una $\beta > 1$.)

Además, para diferenciar $\mathcal{L}_{\mathrm{rec}}$ se menciona el **truco de reparametrización**, donde en vez de muestrear un $z \sim \mathrm{Normal}(\mu, \Sigma)$, se muestrea ruido $\epsilon \sim \mathrm{Normal}(0, I)$ y luego se transforma linealmente

$$z = \mu + \epsilon \cdot \Sigma,$$

de modo que dicho z sea diferenciable.

Nota 3.2.3

> Puedes aprender más sobre el truco de reparametrización en Reparameterization Trick.

El entrenamiento total se puede expresar como el siguiente pseudocódigo:

```
---------------------------
Inicializar φ, θ al azar

Repetir S veces:                    # S = número de pasos de entrenamiento
    Tomar un batch de B tareas

    Para cada tarea del batch:
        Para cada par i:                  # LOO
            Codificar los pares j ≠ i y promediar → z
            Repetir K veces:              # bucle interno: solo se mueve z
                z ← z + α ∇_z Σ_{j≠i} log p_θ(y_j | x_j, z)
            NLL_i = −log p_θ(y_i | x_i, z)
        pérdida de la tarea = Σ_i NLL_i + β Σ_i KL_i

    L_total = Promedio de las pérdidas del batch
    φ ← φ − η ∇_φ L_total                 # bucle externo: 1 actualización
    θ ← θ − η ∇_θ L_total
---------------------------
```

Aquí, los batches son conjuntos colecciones de conjuntos X_m (esto es, pares de ejemplos input-output).

Al terminar estos entrenamientos para ϕ y θ, ambos se dejan fijos durante el funcionamiento del programa (fase de prueba).

Fase de Prueba

Se reciben n pares de ejemplos input-output (los que aparecen en P_m). Pasan por el encoder y sus resultados se promedian para generar z'_0. Luego, se aplica Ascenso de Gradiente para llegar a z'_K (donde K puede ser más grande que en entrenamiento) maximizando la verosi-

militud sobre los n ejemplos. Por último, el decoder recibe x_{n+1} (el nuevo input) y z'_K para producir su respuesta.

Experimentos y Resultados

Los experimentos buscan responder tres preguntas:

1. ¿Puede LPN *mejorar en test-time* buscando en su espacio latente?
2. ¿Cómo afecta la manera de entrenarlo a esa búsqueda?
3. ¿Generaliza fuera de la distribución de entrenamiento?

Nota 3.2.4 Notación de los experimentos

- **Grad N**: se hacen N pasos de ascenso de gradiente en el espacio latente. **Grad 0** significa usar directamente el promedio de los latentes del encoder (sin búsqueda).
- **Sample N**: se muestrean N latentes del encoder y se elige el de mayor log-verosimilitud (búsqueda sin gradientes).
- La misma notación se usa para el **entrenamiento** (cuántos pasos hay en el bucle interno del pseudocódigo) y para la **inferencia** (cuántos pasos se hacen en la fase de prueba). Pueden ser distintos.
- **Grad N ⋆ ⋆**: durante el entrenamiento se deja que el gradiente de θ fluya *a través* del paso de optimización latente (derivadas de segundo orden, como en meta-learning). En la versión normal, el término $\alpha\nabla_z(\dots)$ se trata como una constante (*stop gradient*).
- La métrica es **exact match accuracy**: la predicción cuenta sólo si la grilla (o secuencia) completa es correcta.

Tarea Pattern (tarea de juguete)

Antes de ARC-AGI, los autores estudian la dinámica de LPN en una tarea simple: inputs de 10×10 completamente negros con un pixel azul, que indica dónde el output debe «pegar» un patrón de 4×4. El patrón es el mismo en todos los pares de una especificación (es «el programa»), pero cambia entre especificaciones. Se entrena un modelo de 1M de parámetros por 20k pasos, con 3 semillas.

Entrenamiento	Grad 0	Grad 1	Grad 5	Grad 20	Grad 100	Sample 250
Grad 0	3.2	3.6	18.8	52.5	67.5	3.2
Grad 1	8.6	44.6	85.4	98.4	**99.5**	10.2
Grad 1 ⋆ ⋆	0.6	13.7	60.2	88.9	94.1	0.7
Grad 5	0.0	0.4	31.9	88.5	98.1	0.5
Sample 5	6.1	8.2	27.7	56.3	72.2	6.1

Tabla 1 — Accuracy (%) en la tarea Pattern. Filas: método de entrenamiento. Columnas: método de inferencia.

Lo que se concluye de la tabla:

1. **Sin búsqueda en inferencia, todo falla.** Con Grad 0 en inferencia ninguna variante supera el 9%.
2. **Más búsqueda ⇒ mejor desempeño.**
3. **Entrenar sabiendo que habrá búsqueda importa.**
4. **El gradiente le gana ampliamente al muestreo.**

¿Importa partir la búsqueda desde el encoder?

En la tarea Pattern se compara iniciar la búsqueda latente desde el encoder ($z \sim q_\phi(z|x,y)$) contra iniciarla desde la prior ($z \sim p(z) = \text{Normal}(0, I)$). Iniciar desde el encoder es **crítico** en todos los métodos de entrenamiento.

Nota 3.2.5 Sistema 1 y Sistema 2

> Los autores interpretan esto con la analogía de los dos sistemas de pensamiento (de Kahneman): el encoder hace un razonamiento rápido e intuitivo (**Sistema 1**) que acerca a una buena región del espacio, y la optimización latente hace un razonamiento lento y deliberado (**Sistema 2**) que refina la respuesta. Ninguno de los dos basta por sí solo.

Generalización Fuera de Distribución (OOD)

En la tarea Pattern, los modelos se entrenan con patrones de densidad 50% (mitad negros, mitad de color) y se evalúan con patrones de densidad 100% (*fuertemente OOD*).

Entrenamiento	Grad 0	Grad 10	Grad 100
In-Context	0.0	—	—
TTT	0.0	1.8	0.3
LPN Grad 0	0.3	18.8	41.1
LPN Grad 1	0.0	59.9	**88.0**

Tabla 2 — Accuracy (%) en la tarea Pattern fuertemente OOD.

Ni in-context learning ni TTT (*Test Time Training*, ajustes de los parámetros θ de una red durante el funcionamiento) logran recuperar el patrón correcto. LPN, sin búsqueda, tampoco (0%), pero con 100 pasos de gradiente llega al 88%. Es decir, **la búsqueda en test-time es lo que permite adaptarse a tareas nuevas**.

Composición de Programas

Por último, los autores prueban si la búsqueda permite **componer** operaciones que el modelo sólo vio por separado durante el entrenamiento:

- **Dos operaciones** (extraer el patrón dentro de un recuadro + pintar de rosado los pixeles negros): el encoder sólo extrae el patrón, sin cambiar el color (se sobreajusta a las operaciones aisladas que vio). Con ascenso de gradiente, el latente se ajusta y el decoder produce la composición correcta.
- **Tres operaciones** (una de tipo «gravedad», un relleno y un patrón de tablero de ajedrez para ese relleno): LPN aplica la gravedad y el relleno, pero se queda con el

relleno que vio en entrenamiento y, ni siquiera con búsqueda, logra aplicar el patrón de tablero.

Lo anterior muestra que la búsqueda latente *a veces* permite componer programas en combinaciones nuevas, pero esta capacidad **no es robusta**.

Limitaciones

1. **Generalización composicional limitada.** Como se vio en los experimentos de composición, LPN logra (a veces) componer dos operaciones, pero falla con tres. Cada programa es un **único vector continuo** z, y nada en la arquitectura obliga a que ese vector se pueda descomponer en «partes» reutilizables. Los mismos autores proponen explorar **representaciones discretas** de programas para mejorar esto, que es justamente lo que harán más adelante con NLI (ver la sección *El Aporte de la Discretización*).
2. **Depende del cómputo en test-time.** Sin búsqueda (Grad 0), LPN rinde mal tanto en Pattern como OOD. Es decir, el modelo no «resuelve» la tarea en una pasada: su buen desempeño requiere gastar cómputo en inferencia.
3. **Lejos del estado del arte en ARC.** Su 15.5% OOD está muy por debajo del 75.7% de o3-preview (aunque con costos incomparablemente menores). (Queda abierto cómo escala LPN con más datos, parámetros y cómputo.)

Preguntas

- ¿Cómo evita el sobreajuste?
- No entiendo del todo el flujo al entrenar. ¿Cuándo se usa LOO y cuándo ascenso de gradiente? ¿Se entrenan el encoder y el decoder una sola vez al principio y ya, o se vuelven a entrenar luego de hacer ascenso de grandiente con un ejemplo?
- ¿El promedio en LOO es el que introduce el término de invariante ante permutaciones?
- ¿Qué es el *early stopping*? ¿Por qué ayuda a no sobreajustar?
- ¿Cómo se ahorra costos el modelo?

Glosario

- **Fine-tuning**: Proceso de adaptar los parámetros de una red una vez esta ya ha sido entrenada.
- **Test-time training**: Capacidad de los modelos de aprender (optimizar o hacer búsqueda temporal) luego de ya haber sido «entrenados» (i.e. durante la fase de inferencia), pudiendo ajustarse a los ejemplos de una tarea nueva para resolverla momentáneamente.
- **Few-shot learning**: aprendizaje con pocos casos.

3.3. Gradient-Based Program Synthesis with Neurally Interpreted Languages

Nota 3.3.1 Fuente

> Contenido estudiado de *«Gradient-Based Program Synthesis with Neurally Interpreted Languages»* de M. V. Macfarlane et al. (2026). [arXiv:2604.18907]

Introducen el concepto de **Latent Adaptation Networks (LANs)**, que son redes con encoder y decoder que aprenden un espacio latente para representar el espacio de modelos (o programas). Una característica esencial es que en test-time se busca en el espacio latente para adaptarse y mejorar los resultados.

Los autores presentan la arquitectura **Neural Language Interpreter (NLI)** que justamente es del tipo LAN, pero a diferencia del paper anterior, este aprovechará tokens para construir el programa en el espacio latente (y no representarlo como un único vector), de modo que se tenga mayor poder de generalización. Además, esta característica permite que el modelo pueda resolver problemas más difíciles (que requieren más tokens) que los que se le presentaron durante el entrenamiento.

Formalización

Se formaliza la tarea como un **program induction**. Dado un programa p que genera los pares $S = \{(x_i, y_i)\}_{i=1}^{n}$ (es decir, $p(x_i) = y_i$), buscamos un modelo M que prediga con $M(S, x_{n+1})$, para un nuevo input x_{n+1}, el valor de $p(x_{n+1})$. Se denotará con $\mathcal{P}$ el espacio de posibles programas y se definen distribuciones P_{train} y P_{test} para entrenar y testear, respectivamente, la arquitectura, de modo que se pueda verificar su generalización.

Inferencia Discreta Secuencial

NLI aprende, de los pares input-output, un lenguaje de programación discreto (una especie de propio DSL, que llamaremos **codebook**) y «simbólico» (*symbolic-like*, no definido por humanos) para representar programas como secuencias de tokens (de longitud variable) que son ejecutadas por un ejecutor neuronal *diferenciable*, que también aprendió. (¡Importa mucho que sea diferenciable para la optimización que se realizará!)

Objetivo de Entrenamiento

Para aprender el espacio latente con el que representar problemas, se usa una arquitectura encoder-decoder.

- El encoder, **inductor de programa** q_ϕ, infiere una representación latente de programas desde unos pares input-outputs.
- El decoder, **interpretador neuronal** p_θ, ejecuta los programas del espacio latente para predecir outputs dado un input.

Durante el entrenamiento también se usa la estrategia de *leave-one-out* (LOO) sobre *mini-batches* de tamaño B, donde cada especificación contiene m pares input-output. Así, q_ϕ induce un espacio latente usando $m - 1$ pares y el error de predicción se calcula con aquel par dejado fuera. Definimos, para la b-ésima especificación y el i-ésimo par, $\mathcal{S}_{b,i} := \mathcal{S}_b \setminus$

$\{(x_{b,i}, y_{b,i})\}$. Para cada $\mathcal{S}_{b,i}$ buscamos maximizar la verosimilitud de predecir $y_{b,i}$ a partir de $x_{b,i}$, mientras se regulariza para reutilizar tokens. Así, la función de pérdida a minimizar es

$$\mathcal{L}(\phi, \theta; \mathcal{S}) = \frac{1}{B} \sum_{b=1}^{B} \left(\frac{1}{m} \sum_{i=1}^{m} \mathcal{L}_{\text{recon}}(\phi, \theta; x_{b,i}, y_{b,i}, \mathcal{S}_{b,i}) + \lambda_{\text{reg}} \cdot \mathcal{L}_{\text{reg}}(\phi; \mathcal{S}_b) \right)$$

donde

- $\mathcal{L}_{\text{recon}}$ es la **pérdida de reconstrucción**: asegura que el programa sea lo suficientemente expresivo.

$$\mathcal{L}_{\text{recon}}(\phi, \theta; x, y, S) := -\log p_\theta\big(y \mid x, q_\phi(S)\big).$$

- $\mathcal{L}_{\text{reg}}$ es la **pérdida de regularización del encoder**: motiva la reutilización de tokens ya descubiertos en el vocabulario de tamaño K (¡esto motiva la composicionalidad!).

$$\mathcal{L}_{\text{reg}}(\phi; \mathcal{S}) := \sum_{k=1}^{K} \sum_{j=1}^{T} \left[1 - \exp\left(\sum_{b=1}^{B} \sum_{i=1}^{m} \log\Big(1 - q_\phi^{(k,j)}(\mathcal{S}_{b,i})\Big) \right) \right]$$

  - N: número de posiciones de tokens en el programa.
  - $q_\phi^{(k,j)}(\mathcal{S}_{b,i})$: probabilidad asignada al token k en la posición j por el encoder.

  El $1 - \exp$ consiste en la probabilidad de que el token sea usado al menos una vez en el batch.

El Inductor q_ϕ

El inductor q_ϕ (encoder) se encarga de recibir un elemento de S (un par (x, y)) y mapearlo a $\left(\Delta^{K-1}\right)^T$, donde

$$\Delta^{K-1} := \left\{ x \in \mathbb{R}^K : x_i \geq 0, \sum_{i=1}^{K} x_i = 1 \right\}$$

es el símplex, que consiste en todos aquellos vectores $x \in \mathbb{R}^K$ cuyas entradas determinan una distribución de probabilidad. Es decir, q_ϕ recibe un par (x, y) y lo mapea en un

$$z = (z_1, ..., z_T), \qquad \text{donde cada } z_t \in \Delta^{K-1}.$$

Aquí, cada z es una distribución sobre el *codebook* que ha aprendido el algoritmo.

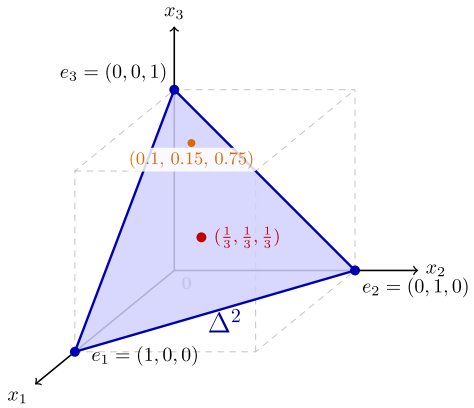


Figura 3 — Gráfico del símplex Δ^2.

Nota 3.3.2

> En el paper además consideran un **skip token** que permite tener programas de largo variable aun teniendo T fijo.

En el entrenamiento,

$$z_t = \text{Softmax}\big(\big(f_\phi(\bar{e}_t) + g_t\big)/\tau_e\big),$$

definiendo

$$\bar{e}_t = \frac{1}{|S|}\sum_i e_t^{(i)}$$

y Softmax es la Gumbel-Softmax con temperatura τ_e en annealing.

El Intérprete p_θ

Para entender cómo funciona el intérprete, es necesaria la siguietne definición.

Definición 3.3.3 Embedder

> Un **embedder** es un algoritmo que transforma datos u objetos discretos a vectores, llamados **embeddings**, y que puede agrupar en una matriz llamada **embedding matrix**.

El intérprete p_θ contiene en θ sus parámetros:

- El **code embedder** E_v (una *embedding matrix*): en su fila k contiende al vector del token número k del *codebook*. Esto permite que la operación $E_v^T z_t$ traduzca la instrucción del paso t a un vector que la red puede leer. Esto es, $E_v^T z_t$ «busca en el codebook».
- El **input-output embedder** E_{io} (otra *embedding matrix*): en su fila k contiene al vector que representa el valor k que puede tomar un elemento de los datos (input y output)
- La red de ejecución d:

Por lo tanto, el intérprete aprende dos matrices de embedding.

La idea del intérprete es que empieza con un estado inicial $s_0 = \text{Embed}(x)$ (el input mismo) y con $\pi_0 = \text{OneHot}(x)$ y en cada paso t actualiza π_t de modo que, de manera muy resumida,

$$s_t = E_{\text{io}}^T \pi_t.$$

Es decir, en cada paso actualiza el estado actual del programa.

Búsqueda en test-time

El objetivo de esta parte es optimizar los embeddings continuos e, no los tokens. Esto es, obtener

$$e^* = \arg\max_e \mathbb{E}_{g,h}\left[\sum_{(x_j,y_j)\in S} \log p_\theta\big(y_j \mid x_j, z(e,\tau_e,g),\tau_d,h\big)\right].$$

Para esto, se parte con temperaturas τ_e, τ_d altas para explorar y se van bajando conforme avanza el tiempo para converger a embeddings discretos.

Resultados

Pseudocódigo del Entrenamiento

```
----------------------------
Inicializar φ, θ al azar

Repetir S veces:                    # S = número de pasos de entrenamiento
    Actualizar τ_e, τ_d             # annealing: τ_e 8 → 0.5, τ_d 2 → 0.5
    Tomar un batch de B tareas (m pares cada una)

    Para cada tarea b del batch:
        Para cada par i:                  # LOO
            # Inductor (encoder)
            Para cada par j ≠ i:
                h_φ(x_j, y_j) → (e_1^(j), ..., e_T^(j))
            Promediar por posición → (ē_1, ..., ē_T)
            Para t = 1, ..., T:
                l_t ← f_φ(ē_t)                    # logits sobre K tokens
                z_t ← Softmax((l_t + g_t) / τ_e)       # g_t ~ Gumbel(0,1)

            # Intérprete (decoder): lee el programa token a token
            s_0 ← Embed(x_i),  π_0 ← OneHot(x_i)
            Para t = 1, ..., T:
                n_t ← d(s_{t-1}, E_v^T z_t)
                π̃t ← Softmax((n_t + h_t) / τ_d)       # h_t ~ Gumbel(0,1)
                π_t ← z_{t,skip} · π_{t-1} + (1 − z_{t,skip}) · π̃t
                s_t ← E_io^T π_t

            NLL_{b,i} = −log π_T(y_i)

    # Regularizador de reuso: nº esperado de tokens distintos en el batch
    L_reg = Σ_k Σ_t [ 1 − Π_{b,i} (1 − q_φ^(k,t)(S_{b,i})) ]

    L_total = (1/B) Σ_b (1/m) Σ_i NLL_{b,i} + λ_reg · L_reg
    φ ← φ − η ∇_φ L_total                 # una sola actualización
    θ ← θ − η ∇_θ L_total
----------------------------
```

Preguntas

- ¿**Test-time** y **tiempo de inferencia** son lo mismo? En este contexto sí. Van contrarios al **training-time**.
- ¿Qué es una **ablación**? Es un experimento del modelo, donde se remueve un componente de él para entender qué tan necesario es.
- ¿Aquí no se usa un muestreo desde una Normal con media μ y varianza Σ?
- ¿$\mathcal{S}_B \subset S$? ¡Sí!
- ¿Qué es un **batch**? Es una porción al azar de datos.

Glosario

- **Transformer**: Arquitectura de red neuronal diseñada específicamente para procesar y generar datos secuenciales, como el lenguaje natural. Consisten en un *encoder* (que convierte la entrada en una matriz) y en un *decoder* (que genera una secuencia final en base a lo que generó el encoder y escogiendo aquella con mayor probabilidad de aparición).

3.4. El Aporte de la Discretización

Aquí discutimos el avance que se realiza desde el LPN al NLI (de, prácticamente, los mismos autores).

Primero, un resumen de las arquitecturas LPN y NLI:

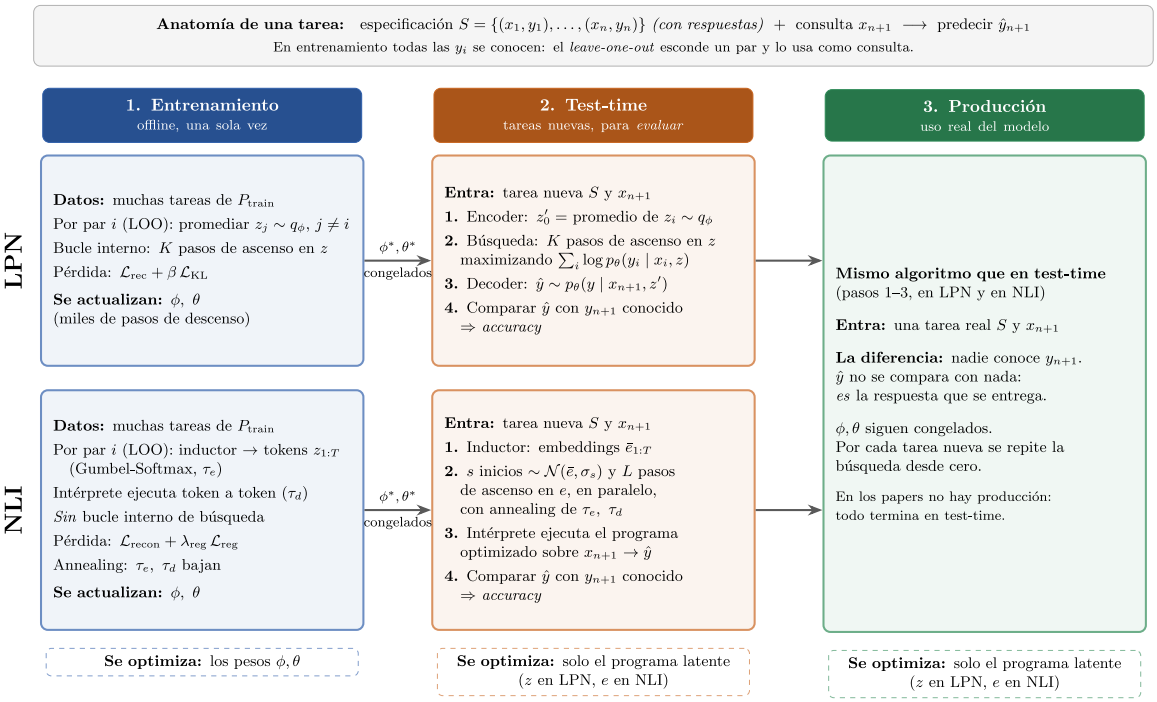


Figura 4 — Relación entre el funcionamiento de entrenamiento y producción de LPN y NLI

El LPN consistía en un encoder y en un decoder. Cuando un ejemplo nuevo x_{n+1} se codificaba, luego se optimizaba por ascenso de gradiente el resultado para mejorarlo (esto dependía del parámetro θ del decoder, por lo que era un proceso determinista una vez ya se había entrenado el θ).

La limitación del LPN era que tenía capacidades limitadas para la generalización composicional. En particular, al componer dos acciones, LPN falla parcialmente, pero al componer tres o más acciones, ya no se logran resultados complacientes. Ante este problema de la composicionalidad, los autores propusieron explorar representaciones latentes discretas para mejorar la generalización composicional, que es lo que justamente hacen en NLI.

En NLI se tiene un encoder que produce tokens discretos usando la Gumbel-Softmax (esto permite extraer muestras «discretas», pero poder seguir realizando *backpropagation*) a la cual se le bajando la temperatura τ con el tiempo. Además, el decoder que usan es especial, pues va leyendo token a token y actualiza un **estado interno** s_t a partir de ellos. Esto permite la **generalización de longitud**: no importa qué tan largo sea el programa, si se sabe interpretar los tokens, cualquier largo finito es ejecutable.

Nota 3.4.1

> NLI también usa ascenso de gradiente luego de que el encoder genere los tokens, sólo que en la Gumbel-Softmax.

En los resultados, se muestra que efectivamente, tanto la discretización como la presencia del estado interno del intérprete permite muchos mejores resultados que LPN.

Hipótesis

La discretización en problemas donde se busca la composicionalidad pareciera ser una mejora natural, pues el pensamiento humano funciona esencialmente en «bloques», en el sentido de que razonamos secuencialmente y en base a conexiones (y muy importante, *composiciones*) entre *ideas* aislada, algo que justamente se puede simbolizar en los computadores por medio de representaciones discretas. Por el contrario, los espacios latentes continuos son más difíciles de relacionar de esta manera con el pensamiento humano.

IV Algoritmos para Redes Neuronales

4.1. Flat Minima

Nota 4.1.1 Fuente

> Contenido estudiado de *«Flat Minima»* de S. Hochreiter y J. Schmidhuber (1997). [DOI:10.1162/neco.1997.9.1.1]

En el paper se presenta un modelo que, además de hacer *backpropagation* para encontrar pesos de una red neuronal, busca mínimos planos que, apoyándose del principio MDL (a partir de robustez), generalicen mejor.

Tarea y Arquitectura

Queremos aproximar una función desconocida $f : X \to Y$, donde $X \subset \mathbb{R}^N$ y $Y \subset \mathbb{R}^K$. Suponga que tenemos un conjunto de datos $D \subset X \times Y$ que se divide en los datos de entrenamiento D_0 y en los datos de test $D \setminus D_0$. Para aproximar f, ajustaremos los pesos $w \in W \subset \mathbb{R}^L$ de la red

$$\mathrm{net}(w) : x \mapsto o(w; x),$$

donde $o(w; x) = (o^1(w; x), ..., o^K(w; x)) \in \mathbb{R}^K$ es la salida de la red con pesos w e input x (es decir, $\mathrm{net}(w)(x) = o(w; x)$).

En el espacio de pesos $W \subset \mathbb{R}^L$ definimos, dado un peso w, la **caja** M_w como un hipercuboide de L dimensiones centrado en w. Además, definimos el **error tolerable** $E_{\mathrm{tol}} > 0$ como el máximo error que aceptaremos respecto a parte de la función de pérdida. Así, un peso w será un **mínimo aceptable** si se satisface que

$$E(\mathrm{net}(w), D_0) := \sum_{(x_p, y_p) \in D_0} \left\| y_p - o\left(w, x_p\right) \right\|^2 \leq E_{\mathrm{tol}},$$

donde $E(\cdot, \cdot)$ es el error cuadrático sumado. (En caso de que la desigualdad sea en sentido contrario hablamos de **underfitting**.) Diremos que un **mínimo plano** es una región conexa de mínimos aceptables. El objetivo del algoritmo será encontrar mínimos aceptables con cajas de gran volumen.

Algoritmo FMS

Se presenta el algoritmo **Flat Minimum Search** (FMS). Este algoritmo no sólo mimiza $E(\mathrm{net}(w), D_0)$ respecto a w, sino que considera

$$E(w, D_0) := E(\mathrm{net}(w), D_0) + \lambda B(w, X_0),$$

donde:

- $X_0 = \{x_p : (x_p, y_p) \in D_0\}$,
- $\lambda \geq 0$ es un hiperparámetro que controla la importancia que se da a que el mínimo sea plano, y
- $B(w, X_0)$ es un término que permite aumentar el volumen de la caja M_w (minimizarlo maximiza el volumen de ella), donde

$$B(w, X_0) = \sum_{x_p \in X_0} B(w, x_p)$$

y

$$B(w, x_p) = \frac{1}{2}\left(-L \log \varepsilon + \sum_{i,j} \log \sum_{k=1}^{K} \left(\frac{\partial o^k(w, x_p)}{\partial w_{ij}} \right)^2 + L \log \sum_{k=1}^{K} \left(\sum_{i,j} \frac{\left| \frac{\partial o^k(w, x_p)}{\partial w_{ij}} \right|}{\sqrt{\sum_{k'=1}^{K} \left(\frac{\partial o^{k'}(w, x_p)}{\partial w_{ij}} \right)^2}} \right)^2 \right).$$

En el siguiente algoritmo, considere

$$E(\text{net}(w), D_0) = \sum_p E_p(w), \qquad \text{donde} \qquad E_p(w) = \left\| y_p - o(w; x_p) \right\|^2.$$

Algoritmo 4.1.2 FMS

Defina un criterio de parada (e.g. un número determinado de épocas, como 400.000). Luego, itere:

- Para cada ejemplo $(x_p, y_p) \in D_0$:
  1. **Pasada forward:** se calcula $o(w; x_p)$.
  2. **Pasada backward:** se hace backpropagation normal, lo que permite obtener
  $$\nabla E_p \quad \text{y} \quad \frac{\partial o^k}{\partial w_{ij}}(w, x_p) \quad (\forall k, w_{ij}).$$
  3. **Gradiente de B.** Se necesitan las segundas derivadas de o^k,
  $$\frac{\partial B}{\partial w_{uv}}(w, x_p) \quad \text{y} \quad \frac{\partial^2 o^k}{\partial w_{ij} \partial w_{uv}}(w, x_p)$$
  y se construye $\nabla B(w, x_p)$.
  4. Se actualizan los pesos
  $$w \leftarrow w - \alpha \nabla E_p - \lambda \cdot \text{escala} \cdot \nabla B,$$
  donde el factor escala fuerza que ∇B y ∇E_p tengan la misma norma.
- Actualice λ según la regla de Weigend et al. (1991) (más grande si el error está bajo E_{tol} o si bajó respecto a la época anterior y más pequeño si el error empeora).

Nota 4.1.3

En el paper, a pesar de que pareciera necesitarse calcular la hessiana (segundas derivadas) de o^k, se usa el truco del operador $\mathcal{R}$ de Pearlmutter para sólo calcular un producto matriz-vector que reduce el costo computacional de $\mathcal{O}(L^2)$ a $\mathcal{O}(L)$.

Notas Adicionales

¿Por qué los mínimos planos generalizan?

Cuando los pesos admiten poca precisión (porque dentro de una caja el error se mantiene acotado), ellos se describen con pocos bits (esto es, bajo valor de B). Por esto, podemos decir que se define una red «simple» (esto es, no es necesario tener pesos tan complejos) y, en consecuencia, se tiene un bajo overfitting esperado gracias al principio MDL.

Nota 4.1.4 MDL

> En la notación de 1.3. Minimum Description Length (MDL), el bajo valor de error corresponde a $L(D|h)$, mientras que el bajo valor de B corresponde a $L(h)$.

Nota 4.1.5 Bajo overfitting esperado

> El describir con pocos bits (esto es, la compresión) permite esperar un bajo overfitting. En el marco del PAC learning (Probably Approximately Correct), existe un argumento formal para esto, que se conoce como **Occam's bound** o **cota de Occam**.

Las dos condiciones de planitud

Se definen dos condiciones para la planitud y que definen la fórmula de B:

- **Condición 1 (robustez a las perturbaciones):** se define una medida $ED(w;\tilde{w})$ que mide cupandi tambia la salida bajo perturbaciones dentro de la caja M_w. Se exige que esta perturbación sea como máximo ε.
- **Condición 2 (igual planitud en todas las direcciones):** se busca que la planitud (el cambio de la salida ante perturbaciones) sea igual para todos los pesos.

Comparación entre Descenso de Gradiente con Backprop y FMS

Los autores aforman que calcular el gradiente

$$\nabla_w(E+\lambda B)$$

cuesta asintóticamente lo mismo que el cálculo de

$$\nabla_w E$$

en el entrenamiento convencional para buscar pesos w de una red neuronal, esto es, cuesta $\mathcal{O}(L)$, donde W corresponde al número de parámetros que tiene la red (número de conexiones entre neuronas, también considerando los sesgos).

Nota 4.1.6

> En verdad, el costo de FMS es $\mathcal{O}(KW)$, pero como consideran el número de salidas de la red, K, constante, lo pueden omitir.

Una diferencia notable (y que se ve en la comparación anterior), es que ambos algoritmos minimizan distintas cantidades: el entrenamiento con backprop sólo se encarga de minimizar $E(w)$, sin importarle la calidad del mínimo (plano o agudo), mientras que a FMS le interesa un mínimo (más bien una zona cercana a un mínimo) donde además haya planitud en todas las direcciones.

En la sección «Multiple initializations?» Hochreiter y Schmidhuber indican que, dado que el backprop convencional siempre prefiere aquellas direcciones donde el gradiente es más empinado, este tiende a preferir zonas que no son planas, por lo que correr el backprop convencional varias veces y escoger la solución más plana no resuelve el problema que ellos intentan resolver.

Nota 4.1.7 Matiz

En el paper *«On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima»*[3] de N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy y P. T. P. Tang (2017). [DOI:10.48550/arXiv.1609.04836], se estudia el SGD a batches e indican que, implícitamente, con batches pequeños el algoritmo prefiere mínimos planos gracias al ruido del gradiente. En este caso, FMS hace esta preferencia explícita y de manera deliberada desde su planteamiento.

[3]Este paper no lo he leído. Es únicamente una precisión que suele hacerse y me ha sido recomendada múltiples veces por distintos LLMs.

V Misceláneos

5.1. DreamerV3

Nota 5.1.1 Fuente

> Contenido estudiado de *«Mastering Diverse Domains through World Models»* de D. Hafner et al. (2023). [arXiv:2301.04104]

Nota 5.1.2

> No confundir con **3.1. DreamCoder**.

DreamerV3 consiste en un algoritmo de RL que aprende un modelo de mundo y le permite al agente imaginar el futuro, escogiendo las acciones que mejores resultados le dan. El modelo supera a otros algoritmos de RL con gran conocimiento de dominio y permite reducir el ajuste especial que se suele tener que hacer a los algoritmos de RL al aplicarlos a una nueva tarea, reduciendo la interacción necesaria.

Los autores afirman que el reto es grande, pues se busca aprender en distintos dominios y con hiperparámetros fijados.

Algoritmo de Aprendizaje

Consiste en tres redes neuronales:

1. El modelo de mundo que predice los resultados de las acciones potenciales.
2. Un juez crítico del valor de cada resultado.
3. Agente que decide los resultados más valiosos.

Estos tres componentes se entrenan de manera concurrente desde experiencia mientras que el agente interactúa con el ambiente.

Aprendizaje del Modelo de Mundo

El modelo aprende representaciones compactas de las entradas sensoriales por medio de *autoencoding*. Se implementa un modelo de mundo como un **Modelo de Estado-Espacio Recurrente** (*Recurrent State-Space Model, RSSM*). En cada paso t:

- Se calcula la *memoria* acumulada de los pasos anteriores,
$$h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}),$$
por medio de un modelo de secuencia (GRU).
- Un encoder mapea los inputs sensoriales x_t a representaciones estocásticas
$$z_t \sim q_\phi(z_t \mid h_t, x_t).$$
- Un predictor intenta reproducir esa misma representación usando sólo la memoria (sin x_t), produciendo
$$\hat{z}_t \sim p_\phi(\hat{z}_t \mid h_t).$$
- La concatenación de h_t y z_t produce el estado del modelo,
$$s_t = \{h_t, z_t\},$$

a partir del cual predecimos las recompensas r_t y las *episode continuation flags* $c_t \in \{0, 1\}$, y reconstruimos los inputs x_t para asegurar representaciones informativas.

Aprendizaje del Crítico

Tanto el actor como el crítico aprenden su comportamiento únicamente a partir del modelo de mundo. Ambos operan en los estados del modelo s_t. El actor apunta a maximizar el retorno

$$R_t = \sum_{\tau=0}^{\infty} \gamma^\tau r_{t+\tau}.$$

Para *rewards* más allá de $T = 16$, el crítico aprende a aproximar la distribución de los retornos para cada estado s_t:

$$\text{Actor:} \quad a_t \sim \pi_\theta(a_t \mid s_t) \qquad \text{Crítico:} \quad v_\psi(R_t \mid s_t).$$

Naturalmente, se leen los valores esperados como los valores predichos por el crítico:

$$v_t = E\big[v_\psi(\cdot \mid s_t)\big] \quad \Big(= E_{R\sim v_\psi(\cdot|s_t)}[R]\Big).$$

Para estimar los retornos que consideran *rewards* más allá del horizonte de predicción, se calculan **boostrapped λ-returns**

$$R_t^\lambda = r_t + \gamma c_t\big((1-\lambda)v_t + \lambda R_{t+1}^\lambda\big)$$

que integran los *rewards* predichos por el crítico con los valores de reward reales, y corresponden al objetivo del crítico.

Nota 5.1.3 On-policy, off-policy

En RL, los datos pueden ser generados:

- por la misma política que actúa, lo que se conoce como que estos son generados **on-policy**;
- por una política distinta a la que actúa, lo que se conoce como **off-policy** (e.g. una política antigua, datasets ya creados [como caso extremo, conocido como **offline RL**])

Aprendizaje del Actor

Queremos que el agente explore más si las recompensas son ralas y explotar más si las recompensas son cercanas. Al mimso tiempo, la cantidad de exploración no debería verse influenciada por escalamientos arbitrarios en recompensas del ambiente. Esto requiere que se normalice la escala de los retornos, mientras se preserva información sobre la frecuencia de las recompensas. Para esto, se normalizan los retornos para que estén aproximadamente en $[0, 1]$ con la cantidad

$$S = \operatorname{EMA}\big(\operatorname{Per}\big(R_t^\lambda, 95\big) - \operatorname{Per}\big(R_t^\lambda, 5\big), 0.99\big),$$

donde EMA corresponde a un «Exponential Moving Average», una función que permite promediar distintos valores, pero dando mayor peso a los valores recientes, y $\alpha = 0.99$ es su decaimiento.

Ciclo de aprendizaje

Como se mencionó, las tres redes aprenden al mismo tiempo aprovechando la experiencia del **replay buffer**.

1. Actuar. El actor juega en el ambiente real y las trayectorias se guardan en el *replay buffer*.
2. Entrenar. En cada paso de gradiente:
    - Se toma un batch de trayectorias reales del buffer.
    - Se actualiza el modelo de mundo con ese batch (pérdidas, etc.).
    - Desde los estados latentes del batch, se imagina con el modelo de mundo actual y el actor actual.
    - Se actualiza el actor y el crítico con las trayectorias imaginadas.
3. Iterar.

Nota 5.1.4

> El **replay buffer** no es más que la memoria que tiene un modelo para poder almacenar datos (especialmente cuando estos son caros).

Resultados

El resultado más destacado es el de lograr obtener diamantes en Minecraft (un entorno muy difícil, por ser grande y aleatorio) en menos de 30 minutos de juego por primera vez sin datos humanos (no como VPT de OpenAI, que usó miles de horas de juego e imitaba el comportamiento humano).

Propiedades de escalamiento. Se mostró en Crafter y DMLab que, con distintas cantidades de parámetros y distintos *replay ratios* (número de veces que se aprovecha una experiencia), hay aprendizaje robusto. Además, aumentar el tamaño del modelo (el número de parámetros) directamente aumenta el rendimiento por tarea y reduce la cantidad de datos requeridos. Asimismo, aumentar el número de pasos de gradiente reduce la interacción necesaria para aprender comportamientos exitosos.

VI Apéndice

Por hacer en el futuro

6.1. Relaciones entre Papers

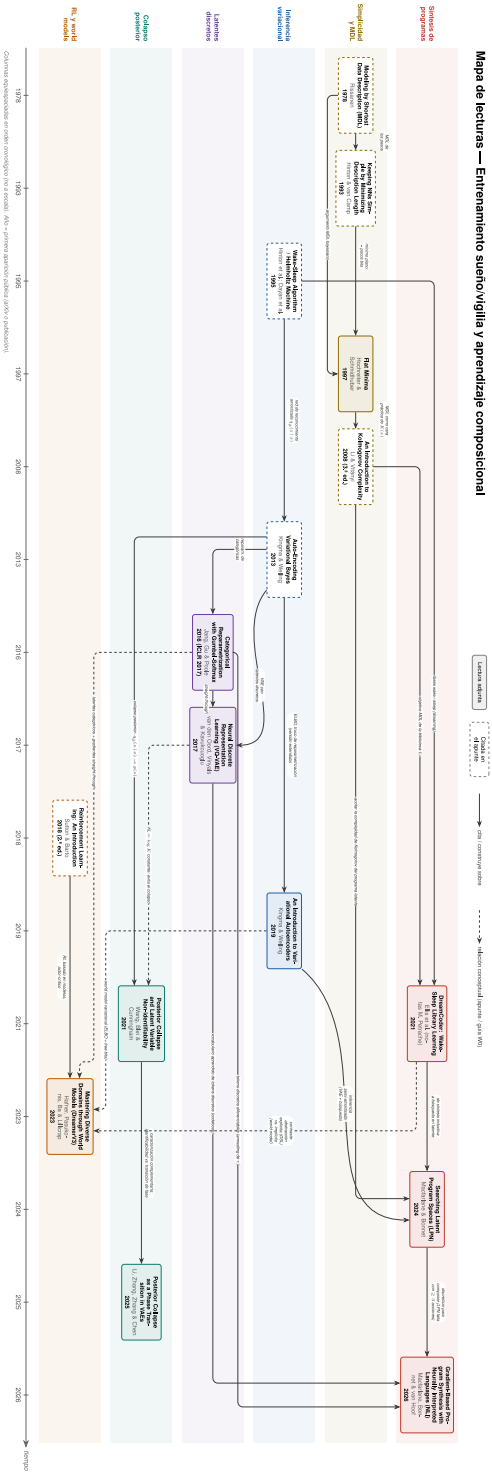


Figura 5 — Mapa de lecturas

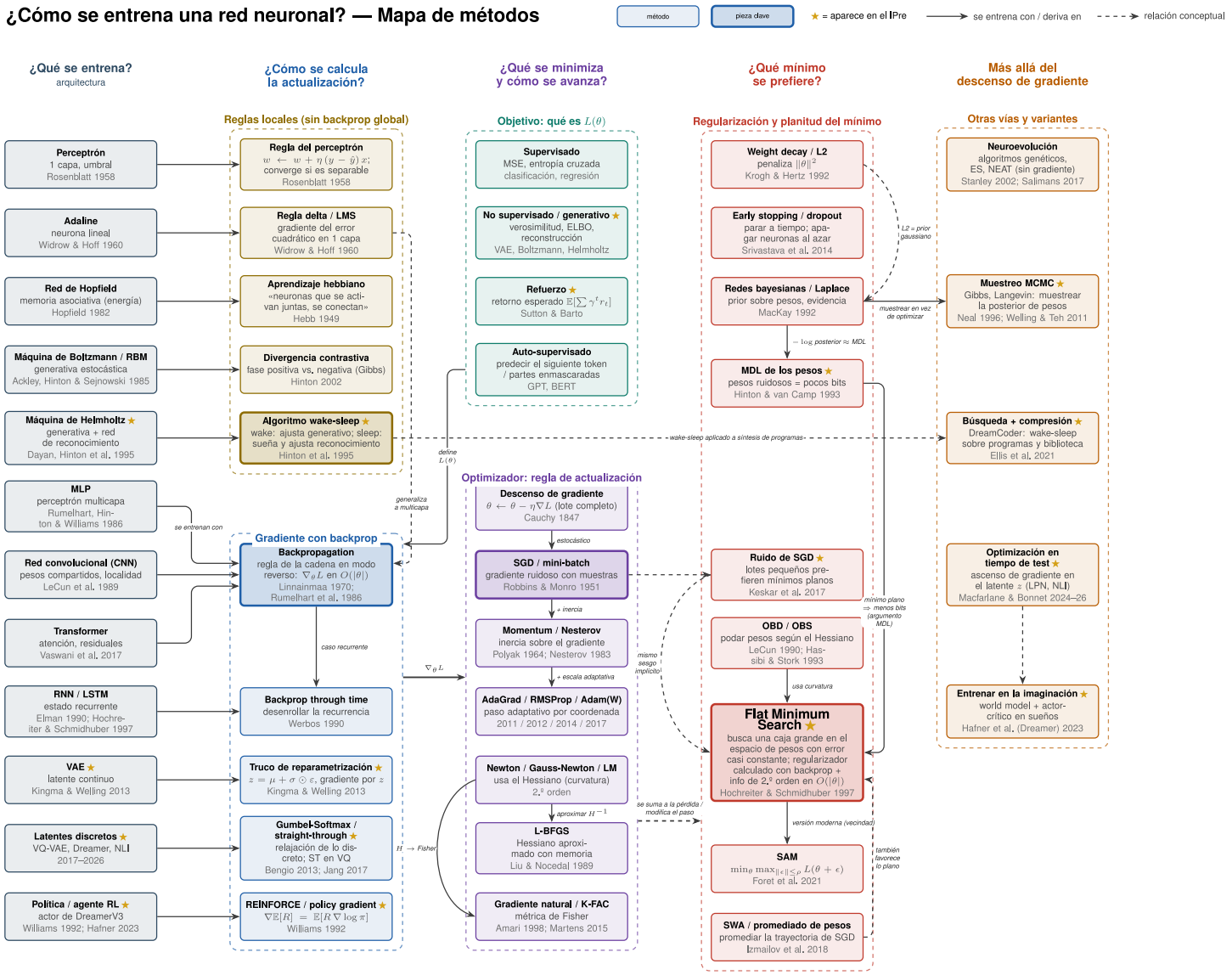


Figura 6 — Mapa de lecturas